

Phase 5: Apex Programming (Developer)

◆ Objective

In this phase, we extend the portal's functionality using **Apex programming**. While earlier phases focused on **data modeling and automation**, this phase introduces **custom logic** for:

- Managing **skills and training (Skill Development)**
- Automating **job applications and employer workflows (Employment)**
- Handling **large-scale data** using asynchronous processing

This ensures that both **Skill Development** and **Employment Management** are deeply integrated into the portal.

◆ Apex Components

1. Apex Classes & Objects

- **SkillManager.cls** → Assigns skills to candidates after training completion.
- **TrainingEnrollment.cls** → Handles candidate enrollment into training programs.
- **JobApplicationHandler.cls** → Validates and processes job applications.
- **EmployerHelper.cls** → Helps recruiters filter candidates by skills and certifications.

Step 1: Implementing `SkillManager.cls`

Purpose in your project

This class will allow you to **assign skills to candidates once they complete a training program**.

For example:

- Candidate Rupa finishes "Java Full Stack" training.
 - System automatically assigns "Java Full Stack" skill to Rupa's profile.
-

✂ Step 1A: Make sure custom objects exist

Before coding, check that these objects exist in your org:

1. **Candidate__c** – stores candidate details (like Name, Email, Phone, etc.)
2. **Skill__c** – stores skill details (like Skill Name, Candidate reference, Certification, etc.)

🔗 Procedure:

- Go to **Setup** → **Object Manager**.
- Check if `Candidate__c` and `Skill__c` exist.
- If not, create them:
- **Candidate__c** → Fields: Name (standard), Email, Phone.

The screenshot shows the Salesforce Object Manager interface for the 'Candidate' object. The left sidebar contains a navigation menu with options: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Restriction Rules, Scoping Rules, Object Access, Triggers, and Flow Triggers. The main content area displays a table of fields for the 'Candidate' object. The table has columns: FIELD LABEL, FIELD NAME, DATA TYPE, CONTROLLING FIE..., and INDEXED. The fields listed are: Candidate Name (Name, Auto Number, indexed), CandidateID (CandidateID__c, Auto Number (External ID), indexed), ContactNumber (ContactNumber__c, Phone), Created By (CreatedById, Lookup(User)), Education (Education__c, Picklist), Email (Email__c, Email (Unique), indexed), Last Modified By (LastModifiedById, Lookup(User)), Place (Place__c, Text(200)), Record Type (RecordTypeId, Record Type, indexed), and Skills (Skills__c, Text Area(255)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIE...	INDEXED
Candidate Name	Name	Auto Number		✓
CandidateID	CandidateID__c	Auto Number (External ID)		✓
ContactNumber	ContactNumber__c	Phone		
Created By	CreatedById	Lookup(User)		
Education	Education__c	Picklist		
Email	Email__c	Email (Unique)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Place	Place__c	Text(200)		
Record Type	RecordTypeId	Record Type		✓
Skills	Skills__c	Text Area(255)		

- **Skill__c** → Fields: Name (standard), Lookup → Candidate__c.

SETUP > OBJECT MANAGER

Skill

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Quick Find

New

Deleted Fields

Field Dependencies

Set History

5 items, sorted by field label

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIE...	INDEXED
Created By	CreatedById	Lookup(User)		
Description	Description__c	Long Text Area(32768)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Proficiency Level	Proficiency_Level__c	Picklist		
Skill Category	Department__c	Picklist		
Skill Name	Skill_Name__c	Text(200)		
Skill Name	Name	Text(80)		✓
Student Name	Student_Name__c	Lookup(Student)		✓

✂ Step 1B: Create Apex Class in Developer Console

1. In Salesforce, click your avatar (top right).
2. Select **Developer Console**.
3. Go to **File** → **New** → **Apex Class**.
4. Enter name: `SkillManager`.

Developer Console - Google Chrome

orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File Edit Debug Test Workspace Help < >

SkillManager.apxc

Code Coverage: None API Version: 64 Go To

```

2
3 public static void assignSkill(Id candidateId, String skillName) {
4     // Fetch Candidate record
5     Candidate__c cand = [
6         SELECT Id, Name
7         FROM Candidate__c
8         WHERE Id = :candidateId
9         LIMIT 1
10    ];
11
12    // Create new Skill record linked to Candidate
13    Skill__c newSkill = new Skill__c(
14        Name = skillName,
15        Candidate__c = cand.Id
16    );
17
18    // Insert record
19    insert newSkill;
20 }
21 }

```

Logs Tests Checkpoints Query Editor View State Progress Problems

User	Application	Operation	Time	Status	Read	Size
T. Rupasree	Unknown	/services/data...	9/21/2025, 1...	Insert failed. ...	Unread	5.26 KB
T. Rupasree	Browser	/setup/build/e...	9/21/2025, 1...	Success	Unread	1.04 KB
T. Rupasree	Browser	/setup/build/e...	9/21/2025, 1...	Success	Unread	1.27 KB
T. Rupasree	Browser	/setup/build/e...	9/21/2025, 1...	Success	Unread	1.27 KB
T. Rupasree	Browser	/setup/build/v...	9/21/2025, 1...	Success	Unread	1.05 KB
T. Rupasree	Browser	/aura	9/21/2025, 1...	Success	Unread	780 bytes

✂ Step 1C: Test the Class (Execute Anonymous Window)

1. In Developer Console → Press **Ctrl + E** (Execute Anonymous).
2. Run this test code:

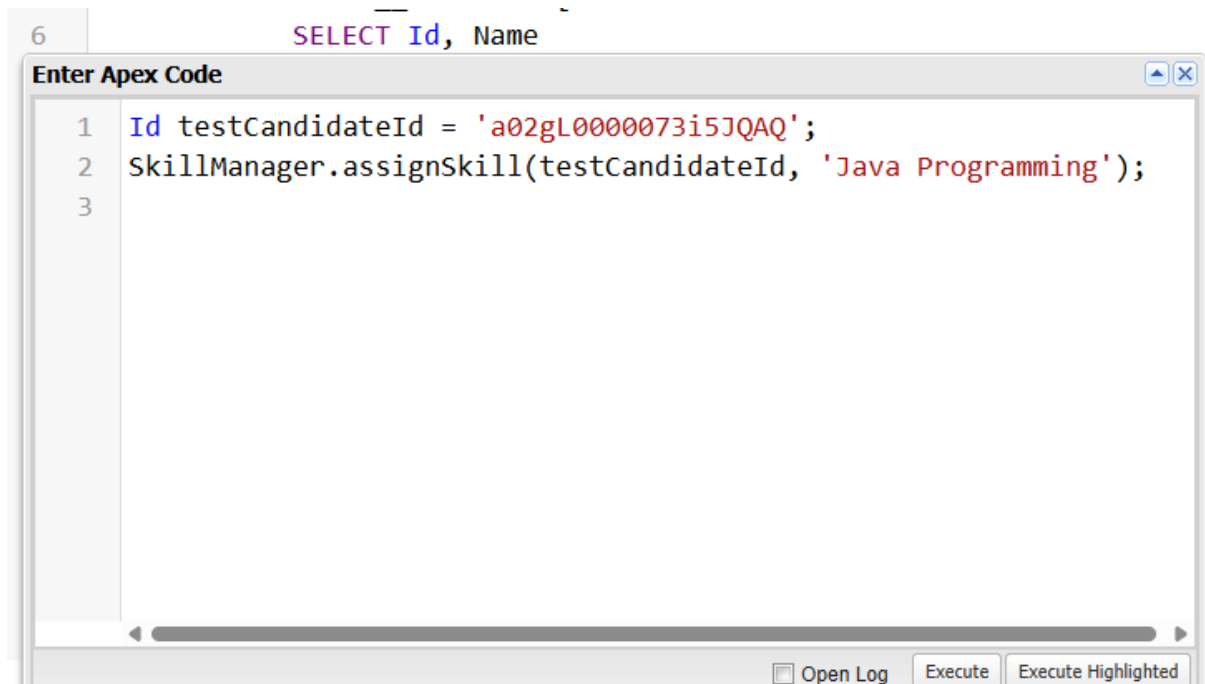
```

// Example: Assign "Java Programming" skill to a Candidate Id
testCandidateId = 'PUT_A_VALID_CANDIDATE_ID_HERE';
SkillManager.assignSkill(testCandidateId, 'Java Programming');

```

🔗 Replace `PUT_A_VALID_CANDIDATE_ID_HERE` with an actual **Candidate__c** Id from your org.

- To find this, go to **App Launcher** → **Candidates** → **open a candidate record** → **copy the Id from URL**.



🔗 Step 1D: Verify the Result

1. Go to **App Launcher** → **Skills** tab.
2. You should see a **new Skill record** created, linked to that Candidate.

Sales Home Opportunities ▾ Leads ▾ Tasks ▾ Files ▾ Accounts ▾ Skills ▾ More ▾

Skill
Java Programming

New Contact Edit New Opportunity ▾

Related **Details**

Skill Name	Owner
Java Programming	T. Rupasree
Skill Name	
Java Programming	
Skill Category	
Proficiency Level	
Description	
Candidate	
CAND-0002	
Created By	Last Modified By
T. Rupasree,	T. Rupasree,
9/20/2025, 11:54 PM	9/20/2025, 11:54 PM

Activity

Filters: All time • All activities • All types

[Refresh](#) • [Expand All](#) • [View All](#)

Upcoming & Overdue

No activities to show.
Get started by sending an email, scheduling a task, and more.

No past activity. Past meetings and tasks marked as done show up here.

2. Apex Triggers

- **CandidateTrigger (before insert)** → Validate unique email & phone.
- **TrainingEnrollmentTrigger (after insert)** → Update candidate's "Skills Acquired" after course completion.
- **JobApplicationTrigger (after update)** → Auto-reject if required skills are missing.
-

Step 1: Verify Objects & Fields

1. Training__c

- Candidate__c → Lookup(Candidate__c)
- Training_Status__c → Picklist (values: Completed, In Progress)
- Training_Name__c → Text

Training

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Object Access

Triggers

Flow Triggers

Validation Rules

Conditional Field
Formatting

Quick Find

New

Deleted Fields

Field Depen

14 items, sorted by Field Label

FIELD LABEL ▲	FIELD NAME	DATA TYPE	CONTROLLIN
Candidate	Candidate__c	Lookup(Candidate)	
Contact	Contact__c	Lookup(Contact)	
Created By	CreatedById	Lookup(User)	
Duration	Duration__c	Number(18, 0)	
End Date	End_Date__c	Date	
Last Modified By	LastModifiedById	Lookup(User)	
Mode	Mode__c	Picklist	
Record Type	RecordTypeId	Record Type	
Start Date	Start_Date__c	Date	
Student Name	Student_Name__c	Master-Detail(Student)	
Trainer Name	Trainer_Name__c	Lookup(Training)	
Training Name	Training_Name__c	Text(200)	
Training Name	Name	Text(80)	

2. Skill__c

- Skill_Name__c → Text (name of skill)
- Candidate__c → Lookup(Candidate__c)

SETUP > OBJECT MANAGER

Skill

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts
List View Button Layout
Restriction Rules
Scoping Rules
Object Access
Triggers
Flow Triggers
Validation Rules
Conditional Field Formatting

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIE...
Candidate	Candidate__c	Lookup(Candidate)	
Created By	CreatedById	Lookup(User)	
Description	Description__c	Long Text Area(32768)	
Last Modified By	LastModifiedById	Lookup(User)	
Owner	OwnerId	Lookup(User,Group)	
Proficiency Level	Proficiency_Level__c	Picklist	
Skill Category	Department__c	Picklist	
Skill Name	Skill_Name__c	Text(200)	
Skill Name	Name	Text(80)	
Student Name	Student_Name__c	Lookup(Student)	

✓ Make sure these fields exist.

Step 2: Create Apex Helper Class

Go to **Setup** → **Apex Classes** → **New** and create `SkillAssignmentHelper.cls`:



SETUP

Apex Classes

Apex Class

SkillAssignmentHelper

Apex Class Detail

Edit

Delete

Download

Security

Show Dependenc

Name	SkillAssignmentHelper	Status	Active
Namespace Prefix		Code Coverage	0% (0/11)
Created By	T. Rupasree , 9/21/2025, 12:32 AM	Last Modified By	T. Rupasree

Class Body

Class Summary

Version Settings

Trace Flags

```

1 public class SkillAssignmentHelper {
2
3     public static void assignSkillsAfterTraining(List<Training__c> newTrainings, Map<Id,
4         List<Skill__c> skillsToInsert = new List<Skill__c>();
5
6     for (Training__c tr : newTrainings) {
7         Training__c oldTr = oldMap.get(tr.Id);
8
9         // Trigger only if status changed to Completed
10        if (tr.Training_Status__c == 'Completed' && oldTr.Training_Status__c != 'Comple
11            Skill__c newSkill = new Skill__c(
12                Skill_Name__c = tr.Training_Name__c,
13                Candidate__c = tr.Candidate__c
14            );
15            skillsToInsert.add(newSkill);
16        }
17    }
18
19    if (!skillsToInsert.isEmpty()) {
20        insert skillsToInsert;
21    }
22 }
23

```

Edit

Delete

Download

Security

Show Dependenc

Step 3: Create the Trigger

- Go to **Setup** → **Object Manager** → **Training__c** → **Triggers** → **New**
- Enter:

```
trigger AssignSkillAfterTraining on Training__c (after update) {
    SkillAssignmentHelper.assignSkillsAfterTraining(Tri
    Trigger.new,
    Trigger.oldMap); }
```

✓ This ensures your trigger is **bulk-safe** and clean.

Apex Trigger
AssignSkillAfterTraining

[Help for this Page](#) 

Apex Trigger Edit

Save

Quick Save

Cancel

Apex Trigger

Version Settings

is active ☒

```
1 trigger AssignSkillAfterTraining on Training__c (after  
2     SkillAssignmentHelper.assignSkillsAfterTraining(T  
3 }
```

Position: Ln 3, Ch 2 Total: Ln 3, Ch 146

Step 4: Test the Trigger

1. Open a **Training__c** record for a candidate.

 Training

Verbal Communications

Related

Details

Training Name

Verbal Communications

Training Name

Verbal Communications

Contact

Duration

2

Mode

Offline

Start Date

9/4/2025

End Date

11/4/2025

Candidate

[CAND-0001](#)

Training_Status

In Progress

Student Name

[Manoj](#)

Created By

 T. Rupasree ,
9/21/2025, 1:46 AM

Last Modified By

 T. Rupasree ,
9/21/2025, 1:51 AM

- Change **Training_Status_c** from **In Progress** → **Completed**.



Training

Verbal Communications

Related

Details

Training Name

Verbal Communications

Training Name

Verbal Communications

Contact

Duration

2

Mode

Offline

Start Date

9/4/2025

End Date

11/4/2025

Candidate

CAND-0001

Training_Status

Completed

Student Name

Manoj

Created By



T. Rupasree,

9/21/2025, 1:46 AM

Last Modified By



T. Rupasree,

9/21/2025, 1:51 AM

- Save the record.
- Go to the candidate record → check the **Skills related list**. You should see a new **Skill_c** record created automatically.

SEARCH...

SalesHomeOpportunitiesLeadsTasksFilesSkillsMore

Skill

a08gL00000AVJIB

New Contact

Edit

New Opportunity

Related

Details

Skill Name

a08gL00000AVJIB

Owner

T. Rupasree

Skill Name

Verbal Communications

Skill Category

Proficiency Level

Description

Candidate

CAND-0001

Created By

T. Rupasree,

9/21/2025, 1:48 AM

Last Modified By

T. Rupasree,

9/21/2025, 1:48 AM

Activity

Filters: All time • All activities • All types

Refresh

Expand All

View All

Upcoming & Overdue

No activities to show.

Get started by sending an email, scheduling a task, and more.

No past activity. Past meetings and tasks marked as done show up here.

Step 5: Test Class (for Deployment)


Apex Classes

Apex Class
SkillAssignmentHelperTest
[Help for this Page ?](#)

Edit
Delete
Download
Run Test

Show Dependencies

Name	SkillAssignmentHelperTest	Status	Active
Namespace Prefix		Created By	T. Rupasree , 9/21/2025, 1:54 AM
Last Modified By	T. Rupasree , 9/21/2025, 1:54 AM		

Class Body

Class Summary

Version Settings

Trace Flags

```

1  @isTest
2  public class SkillAssignmentHelperTest {
3
4      @isTest static void testAssignSkill() {
5
6          // 1 Create a Candidate record
7          Candidate__c cand = new Candidate__c(
8              /* Only set required fields, skip Name if auto-number */
9          );
10         insert cand;
11
12         // 2 Create a Training__c record
13         Training__c tr = new Training__c(
14             Candidate__c = cand.Id,
15             Training_Status__c = 'In Progress',
16             Training_Name__c = 'Java Programming'
17         );
18         insert tr;
19
20         // 3 Update the Training__c status to Completed
21         tr.Training_Status__c = 'Completed';
22         update tr;
23
24         // 4 Verify Skill__c creation
25         List<Skill__c> skills = [
26             SELECT Skill_Name__c, Candidate__c
27             FROM Skill__c
28             WHERE Candidate__c = :cand.Id
29         ];
30
31         System.assertEquals(1, skills.size());
32         System.assertEquals('Java Programming', skills[0].Skill_Name__c);
33     }
34 }

```

Summary

1. Using **Training__c** instead of Training_Enrollment__c.
2. Using **Skill_Name__c** instead of Name for Skill__c.
3. Trigger fires **after update** and inserts skills when training is completed.

3. Trigger Design Pattern

A **Trigger Design Pattern** is a way to structure your Apex triggers so they are:

- **Bulk-safe** – handles multiple records at once
- **Readable and maintainable** – logic is separated from the trigger
- **Reusable** – can call helper classes for different operations

Why it's needed:

Without a design pattern, your triggers can become messy and cause:

- DML limits errors
- Hard-to-maintain code
- Duplicate logic

2 Common Trigger Design Pattern Structure

1. **Trigger** – only contains calls to a **handler class**
2. **Handler class** – contains methods like `beforeInsert`, `afterInsert`, `beforeUpdate`, `afterUpdate`
3. **Separation of concerns** – triggers don't directly perform business logic

Example Structure:

```
// Trigger trigger TrainingTrigger on Training__c (before insert, after
update) { if(Trigger.isBefore && Trigger.isInsert) {
TrainingHandler.beforeInsert(Trigger.new); } if(Trigger.isAfter &&
Trigger.isUpdate) { TrainingHandler.afterUpdate(Trigger.new,
Trigger.oldMap); } } // Handler Class public class TrainingHandler { public
static void beforeInsert(List<Training__c> newTrainings) { // Logic before
insert } public static void afterUpdate(List<Training__c> updatedTrainings,
Map<Id, Training__c> oldMap) { for(Training__c t : updatedTrainings) {
Training__c oldT = oldMap.get(t.Id); if(t.Training_Status__c == 'Completed'
&& oldT.Training_Status__c != 'Completed') { Skill__c skill = new Skill__c(
Candidate__c = t.Candidate__c, Skill_Name__c = t.Training_Name__c ); insert
skill; } } } }
```

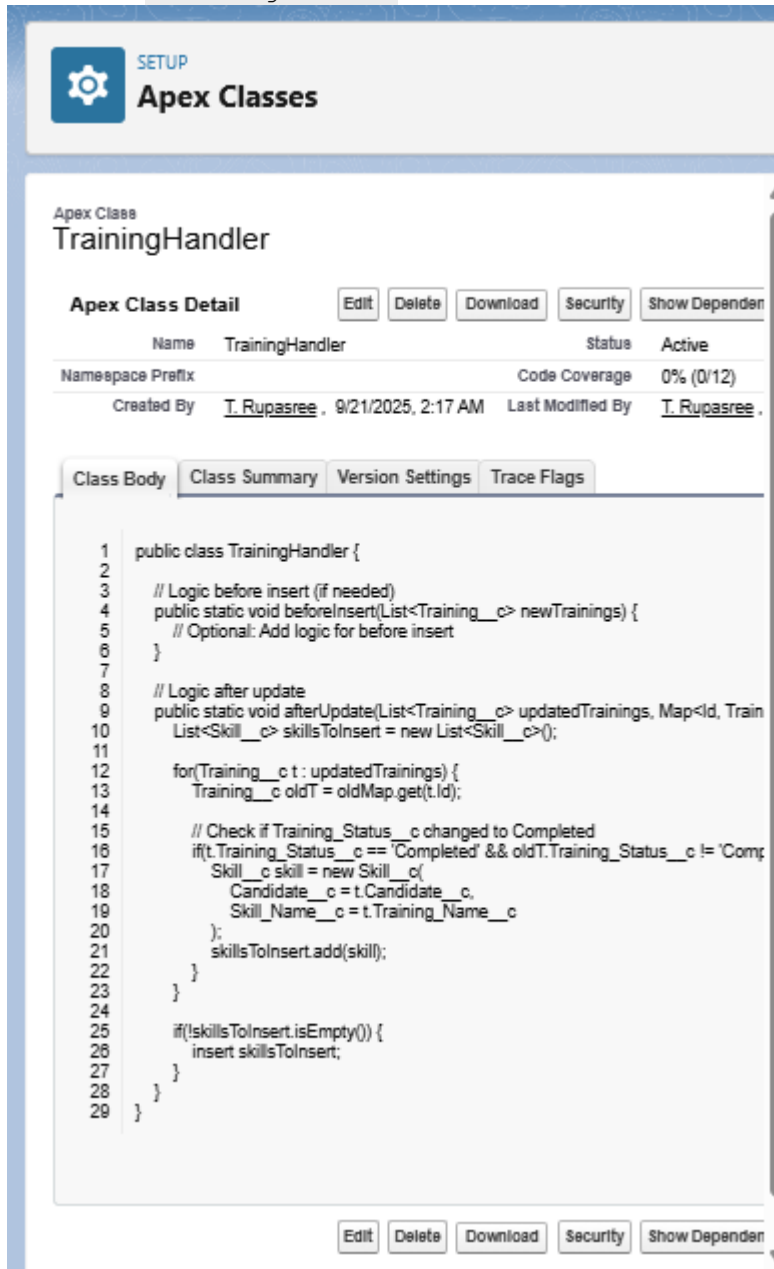
3 Benefits

- **Bulk-safe:** handles multiple Training__c records at once
- **Easy to maintain:** business logic is in the handler, not in the trigger
- **Scalable:** easy to extend for other objects like Job applications

4 Implement in our Project

1. Create the Handler Class

- Go to **Setup** → **Apex Classes** → **New**
- Name it `TrainingHandler`
- Paste the `TrainingHandler` code of it



The screenshot displays the Salesforce Apex Class Editor interface for a class named `TrainingHandler`. The top navigation bar includes the 'SETUP' icon and the 'Apex Classes' title. Below this, the class name 'TrainingHandler' is shown. A table provides details about the class, including its name, namespace prefix, status (Active), code coverage (0% of 12), and creation/modification information by user T. Rupasree. The 'Class Body' tab is selected, showing the following Apex code:

```
1 public class TrainingHandler {
2
3     // Logic before insert (if needed)
4     public static void beforeInsert(List<Training__c> newTrainings) {
5         // Optional: Add logic for before insert
6     }
7
8     // Logic after update
9     public static void afterUpdate(List<Training__c> updatedTrainings, Map<Id, Train
10     List<Skill__c> skillsToInsert = new List<Skill__c>();
11
12     for(Training__c t : updatedTrainings) {
13         Training__c oldT = oldMap.get(t.Id);
14
15         // Check if Training_Status__c changed to Completed
16         if(t.Training_Status__c == 'Completed' && oldT.Training_Status__c != 'Comp
17             Skill__c skill = new Skill__c(
18                 Candidate__c = t.Candidate__c,
19                 Skill_Name__c = t.Training_Name__c
20             );
21             skillsToInsert.add(skill);
22         }
23     }
24
25     if(!skillsToInsert.isEmpty()) {
26         insert skillsToInsert;
27     }
28 }
29 }
```

At the bottom of the editor, there are buttons for 'Edit', 'Delete', 'Download', 'Security', and 'Show Dependencies'.

2. Create the Trigger

- Go to **Setup** → **Object Manager** → **Training__c** → **Triggers** → **New**
- Name it `TrainingTrigger`
- Paste the trigger code



SETUP

Apex Triggers

Apex Trigger

TrainingTrigger

Help for this Page ?

Apex Trigger Detail

Edit

Delete

Download

Show Dependencies

Name	TrainingTrigger	sObject Type	Training
Code Coverage	0% (0/4)	Status	Active
Created By	T. Rupasree, 9/21/2025, 2:18 AM	Last Modified By	T. Rupasree, 9/21/2025, 2:18 AM
Namespace Prefix			

Apex Trigger

Version Settings

Trace Flags

```

1 trigger TrainingTrigger on Training__c (before insert, after update) {
2
3     if (Trigger.isBefore && Trigger.isInsert) {
4         TrainingHandler.beforeInsert(Trigger.new);
5     }
6
7     if (Trigger.isAfter && Trigger.isUpdate) {
8         TrainingHandler.afterUpdate(Trigger.new, Trigger.oldMap);
9     }
10 }

```

Edit

Delete

Download

Show Dependencies

3. Test the Trigger

- Use the **anonymous Apex script** we wrote earlier, or the test class
- Update `Training_Status__c` to `Completed`
- Verify a `Skill__c` record is automatically created



Candidate
CAND-0001

Related

Details


Skills (2)

New

Skill Name

a08gL00000AVJIB

a08gL00000AVWYv

View All


Trainings (1)

New

Training Name

Verbal Communications

View All

trigger TrainingTrigger on Training__c (before insert, after update)

```

if (Trigger.isBefore && Trigger.isInsert) {
    TrainingHandler.beforeInsert(Trigger.new);
}

if (
}

```

Enter Apex Code

```

1 // Create a candidate
2 Candidate__c cand = new Candidate__c(
3     Email__c = 'test@example.com',
4     ContactNumber__c = '9999999999',
5     Place__c = 'Nagari',
6     Education__c = 'B Tech'
7 );
8 insert cand;
9
10 // Create a Training record
11 Training__c tr = new Training__c(
12     Candidate__c = cand.Id,
13     Training Status__c = 'In Progress'

```

Open Log

Execute

Execute Highlighted

- All triggers delegate logic to **Handler Classes**.

- **Example:** `JobApplicationTrigger` → `JobApplicationHandler.cls`.
 - Ensures scalability and easy maintenance.
-

4. SOQL & SOSL Queries

- **SOQL:** Fetch candidates with specific skills for a given job.
- **SOSL:** Search courses or certifications across multiple objects quickly.

SOQL lets you fetch records from your custom objects (Candidate, Skill, Job, Training).

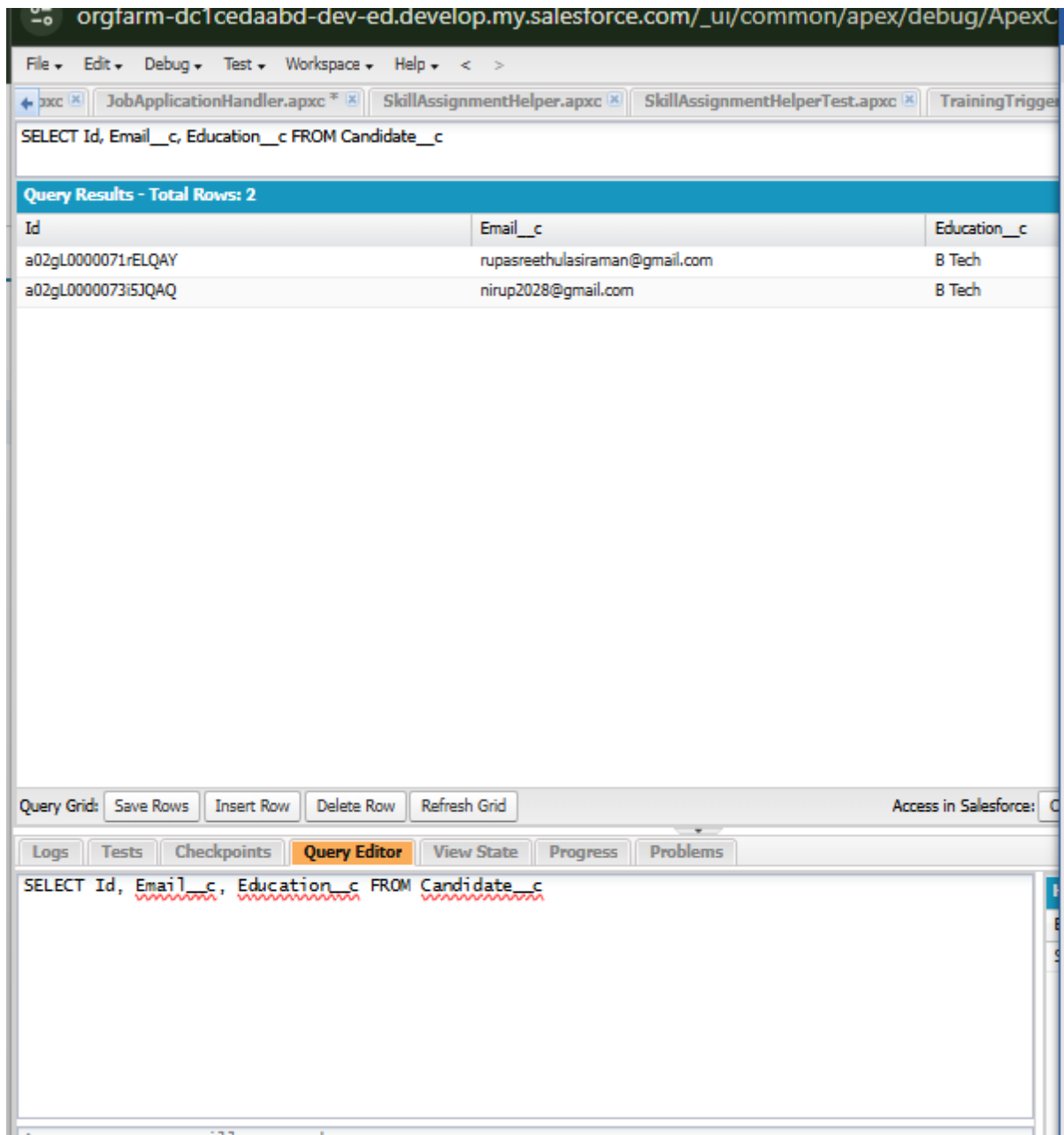
Step 1: Open Developer Console

1. Log in to Salesforce.
 2. In the top-right corner → Click on your avatar → **Developer Console**.
 3. Developer Console opens in a new window.
-

Step 2: Run a Simple SOQL Query

1. In Developer Console → Go to **Query Editor** (bottom tab).
2. Type this query:

```
SELECT Id, Email__c, Education__c FROM Candidate__c
```
3. Click **Execute**.
4. You will now see all **Candidate records** with their Email & Education in a table.



Step 3: Run SOQL Inside Apex (Execute Anonymous)

1. In Developer Console → **Debug** → **Open Execute Anonymous Window (Ctrl+E)**.
2. Paste this code:

```
List<Candidate__c> candidates = [ SELECT Id, Email__c, Education__c, Place__c FROM Candidate__c ]; System.debug('Candidate Records: ' + candidates);
```
3. Tick ☒ **Open Log** → Click **Execute**.

Enter Apex Code

```

1 List<Candidate__c> candidates = [
2     SELECT Id, Email__c, Education__c, Place__c
3     FROM Candidate__c
4 ];
5
6 System.debug('Candidate Records: ' + candidates);
7

```

☒ Open Log
 Execute

- In the log window → Search for **USER_DEBUG** → you'll see candidate records printed.

File Edit Debug Test Workspace Help < >		
TestHelper.apxc SkillAssignmentHelperTest.apxc TrainingTrigger.apxt Candidate__c@3:27 PM Log executeAnonymous @9/21/2025, 3:31:06 PM		
Execution Log		
Timestamp	Event	Details
15:31:06:001	USER_INFO	[EXTERNAL] 005gL000005Dp8H rupasree287375@ageantforce.com (GMT-07:00) Pacific Daylight Time (America/Los_Angeles) GMT-07:00
15:31:06:001	EXECUTION_ST...	
15:31:06:001	CODE_UNIT_ST...	[EXTERNAL] execute_anonymous_apex
15:31:06:001	VARIABLE_SCO...	[1] candidates List<Candidate__c> true false
15:31:06:001	HEAP_ALLOCATE	[95] Bytes:3
15:31:06:001	HEAP_ALLOCATE	[100] Bytes:152
15:31:06:001	HEAP_ALLOCATE	[417] Bytes:408
15:31:06:001	HEAP_ALLOCATE	[430] Bytes:408
15:31:06:001	HEAP_ALLOCATE	[317] Bytes:6
15:31:06:001	HEAP_ALLOCATE	[EXTERNAL] Bytes:3
15:31:06:002	STATEMENT_EX...	[1]
15:31:06:002	STATEMENT_EX...	[1]
15:31:06:002	HEAP_ALLOCATE	[1] Bytes:61
15:31:06:002	HEAP_ALLOCATE	[1] Bytes:4
15:31:06:002	HEAP_ALLOCATE	[68] Bytes:5
15:31:06:002	HEAP_ALLOCATE	[74] Bytes:5
15:31:06:002	HEAP_ALLOCATE	[82] Bytes:7
15:31:06:002	SOQL_EXECUTE...	[1] Aggregations:0 SELECT Id, Email__c, Education__c, Place__c FROM Candidate__c
15:31:06:011	SOQL_EXECUTE...	[1] Rows:2
15:31:06:011	HEAP_ALLOCATE	[1] Bytes:12
15:31:06:011	HEAP_ALLOCATE	[1] Bytes:279
15:31:06:011	HEAP_ALLOCATE	[1] Bytes:12
15:31:06:011	VARIABLE_ASSI...	[1] candidates [{RecordTypeId:"012gL000002UWHRQA4","Id":"a02gL0000071rELQAY","Education__c":"B Tech","Place__c":"Nagari","Email__c":"rupa
15:31:06:011	STATEMENT_EX...	[6]
15:31:06:011	HEAP_ALLOCATE	[6] Bytes:19
15:31:06:011	HEAP_ALLOCATE	[6] Bytes:289
15:31:06:011	HEAP_ALLOCATE	[6] Bytes:308
15:31:06:011	USER_DEBUG	[6] DEBUG Candidate Records: (Candidate__c:{Id=a02gL0000071rELQAY, Email__c=rupasreethulasiraman@gmail.com, Education__c=B Tech, Place__
15:31:06:012	CUMULATIVE_L...	
15:31:06:012	LIMIT_USAGE...	(default)
15:31:06:000	LIMIT_USAGE...	Number of SOQL queries: 1 out of 100

Step 4: SOQL with Condition

Example: Get all candidates from **Nagari**:

```
List<Candidate__c> candList = [ SELECT Id, Email__c, Education__c, Place__c  
FROM Candidate__c WHERE Place__c = 'Nagari' ]; System.debug('Candidates  
from Nagari: ' + candList);
```

Enter Apex Code

```
1 List<Candidate__c> candidates = [  
2     SELECT Id, Email__c, Education__c, Place__c  
3     FROM Candidate__c  
4 ];  
5  
6 System.debug('Candidate Records: ' + candidates);  
7
```

☒ Open Log

Execute

Execute Highlighted

ILE_ASSL... [1]|candidates|[{ "RecordTypeId": "012gL000000Z0WHRQA4", "Id": "a02gL0000001rELQAY", "Education__c": "B Tech", "Place__c": "Nagari", "Emi

Execution Log		
Timestamp	Event	Details
15:32:40:002	USER_INFO	[EXTERNAL] [005gL000005Dp8H rupasree287375@ageantforce.com (GMT-07:00) Pacific Daylight Time (America/Los_Angeles) GMT-07:00
15:32:40:002	EXECUTION_ST...	
15:32:40:002	CODE_UNIT_ST...	[EXTERNAL] execute_anonymous_apex
15:32:40:002	VARIABLE_SCO...	[1] candidates List<Candidate__c> true false
15:32:40:002	HEAP_ALLOCATE	[95] Bytes:3
15:32:40:002	HEAP_ALLOCATE	[100] Bytes:152
15:32:40:002	HEAP_ALLOCATE	[417] Bytes:408
15:32:40:002	HEAP_ALLOCATE	[430] Bytes:408
15:32:40:002	HEAP_ALLOCATE	[317] Bytes:6
15:32:40:002	HEAP_ALLOCATE	[EXTERNAL] Bytes:3
15:32:40:002	STATEMENT_EX...	[1]
15:32:40:002	STATEMENT_EX...	[1]
15:32:40:002	HEAP_ALLOCATE	[1] Bytes:61
15:32:40:002	HEAP_ALLOCATE	[1] Bytes:4
15:32:40:002	HEAP_ALLOCATE	[68] Bytes:5
15:32:40:002	HEAP_ALLOCATE	[74] Bytes:5
15:32:40:002	HEAP_ALLOCATE	[82] Bytes:7
15:32:40:003	SOQL_EXECUTE...	[1] Aggregations:0 SELECT Id, Email__c, Education__c, Place__c FROM Candidate__c
15:32:40:012	SOQL_EXECUTE...	[1] Rows:2
15:32:40:012	HEAP_ALLOCATE	[1] Bytes:12
15:32:40:013	HEAP_ALLOCATE	[1] Bytes:279
15:32:40:013	HEAP_ALLOCATE	[1] Bytes:12
15:32:40:013	VARIABLE_ASSI...	[1] candidates [{"RecordTypeId":"012gL000002UWHRQA4","Id":"a02gL0000071rELQAY","Education__c":"B Tech","Place__c":"Nagari","Email__c":"rupa
15:32:40:013	STATEMENT_EX...	[6]
15:32:40:013	HEAP_ALLOCATE	[6] Bytes:19
15:32:40:013	HEAP_ALLOCATE	[6] Bytes:289
15:32:40:013	HEAP_ALLOCATE	[6] Bytes:308
15:32:40:013	USER_DEBUG	[6] DEBUG Candidate Records: (Candidate__c:[Id=a02gL0000071rELQAY, Email__c=rupasreethulasiraman@gmail.com, Education__c=B Tech, Place__c
15:32:40:013	CUMULATIVE_L...	
15:32:40:013	LIMIT_USAGE_...	(default)
15:32:40:000	LIMIT_USAGE_...	Number of SOQL queries: 1 out of 100

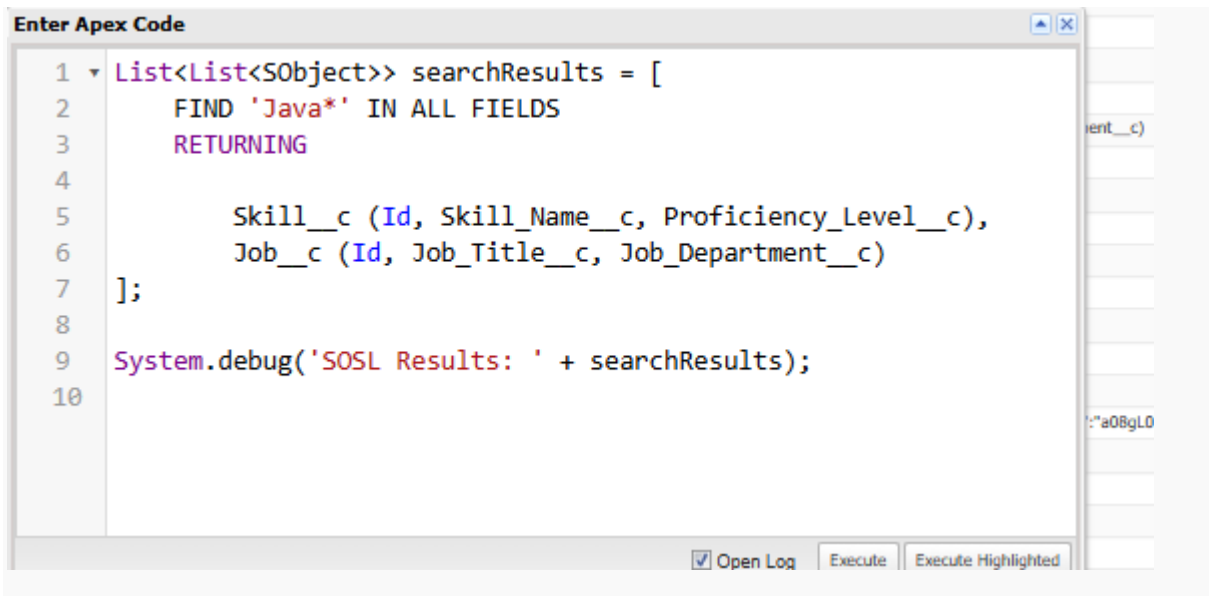
◆ Step: Implementing SOSL

SOSL is used when you want to **search across multiple objects at once** (Candidate, Skill, Job).

Step 1: Execute SOSL in Developer Console

1. Open **Execute Anonymous (Ctrl+E)**.
2. Paste this code:

```
List<List<SObject>> searchResults = [ FIND 'Java*' IN ALL FIELDS RETURNING
Candidate__c (Id, Email__c, Education__c), Skill__c (Id, Skill_Name__c,
Proficiency_Level__c), Job__c (Id, Job_Title__c, Job_Department__c) ];
System.debug('SOSL Results: ' + searchResults);
```



3. Click **Execute with Open Log**.
4. In the log, search **USER_DEBUG** → You'll see results like:

15:35:10:282	HEAP_ALLOCATE	[1] Bytes:12
15:35:10:283	VARIABLE_ASSI...	[1]searchResults[[{"Skill_Name__c":"Java Programming","Id":"a08gl.00000AVKBCQAX"},{"Skill_Name__c":"Java Programming","Id":"a08gl.00000AVZ7QAP"},{"Skill_Name__c":"Java Full Stack","Id":"a08gl.00000AVH9mQAH"}]]]](0x57d5ec60
15:35:10:283	STATEMENT_EX...	[9]
15:35:10:283	HEAP_ALLOCATE	[9] Bytes:14
15:35:10:283	HEAP_ALLOCATE	[9] Bytes:303
15:35:10:283	HEAP_ALLOCATE	[9] Bytes:217
15:35:10:283	USER_DEBUG	[9]DEBUG[SOSL Results: ((Skill__c:(Id=a08gl.00000AVKBCQAX, Skill_Name__c=Java Programming), Skill__c:(Id=a08gl.00000AVZ7QAP, Skill_Name__c=Java Programming), Skill__c:(Id=a08gl.00000AVH9mQAH, Skill_Name__c=Java Full Stack)), (]
15:35:10:283	CLAIM ATTACH...	
15:35:10:283	LIMIT_USAGE...	(default)]
15:35:10:000	LIMIT_USAGE...	Number of SOQL queries: 0 out of 100
15:35:10:000	LIMIT_USAGE...	Number of query rows: 0 out of 50000
15:35:10:000	LIMIT_USAGE...	Number of SOSL queries: 1 out of 20

- Candidates who have "Java" in their Education/Email.
- Skills with "Java" in Skill_Name__c.
- Jobs with "Java" in Job_Title__c or Department.

Step: Apply to our Project

Example Use Cases:

1. Find Jobs Matching Candidate Skills

```

Id candId = 'PUT_CANDIDATE_ID_HERE'; List<Skill__c> skills = [ SELECT
Skill_Name__c FROM Skill__c WHERE Candidate__c = :candId ]; Set<String>
skillNames = new Set<String>(); for (Skill__c s : skills) {
skillNames.add(s.Skill_Name__c); } List<Job__c> matchingJobs = [ SELECT Id,
Job_Title__c, Job_Department__c, salary_range__c FROM Job__c WHERE
Job_Department__c IN :skillNames ]; System.debug('Matching Jobs: ' +
matchingJobs);

```

Enter Apex Code

```

1 Id candId = 'a02gL0000071rELQAY';
2 List<Skill__c> skills = [
3     SELECT Skill_Name__c
4     FROM Skill__c
5     WHERE Candidate__c = :candId
6 ];
7
8 Set<String> skillNames = new Set<String>();
9 for (Skill__c s : skills) {
10     skillNames.add(s.Skill_Name__c);
11 }
12
13 List<Job__c> matchingJobs = [
14     SELECT Id, Job_Title__c, Job_Department__c, salary_range__c
15     FROM Job__c
16     WHERE Job_Department__c IN :skillNames
17 ];
18
19 System.debug('Matching Jobs: ' + matchingJobs);
20

```

[Open Log](#) [Execute](#) [Execute Highlighted](#)
[Anonymous @9/21/2025, 3:32:40 PM](#) [Log executeAnonymous @9/21/2025, 3:35:10 PM](#) [Log executeAnonymous @9/21/2025, 3:39:50 PM](#)

Execution Log

Timestamp	Event	Details
15:39:50:002	HEAP_ALLOCATE	[1]Bytes:4
15:39:50:002	VARIABLE_ASSI...	[1] candId "a02gL0000071rELQAY"
15:39:50:002	STATEMENT_EX...	[2]
15:39:50:002	HEAP_ALLOCATE	[2]Bytes:64
15:39:50:002	HEAP_ALLOCATE	[2]Bytes:4
15:39:50:002	HEAP_ALLOCATE	[2]Bytes:7
15:39:50:002	HEAP_ALLOCATE	[68]Bytes:5
15:39:50:002	HEAP_ALLOCATE	[74]Bytes:5
15:39:50:002	HEAP_ALLOCATE	[82]Bytes:7
15:39:50:003	SOQL_EXECUTE...	[2] Aggregations:0 SELECT Skill_Name__c FROM Skill__c WHERE Candidate__c = :tmpVar1
15:39:50:016	SOQL_EXECUTE...	[2] Rows:2
15:39:50:016	HEAP_ALLOCATE	[2]Bytes:12
15:39:50:016	HEAP_ALLOCATE	[2]Bytes:134
15:39:50:016	HEAP_ALLOCATE	[2]Bytes:12
15:39:50:017	VARIABLE_ASSI...	[2] skills [{{"Skill_Name__c":"Verbal Communication (1 more) ...","Id":"a08gL00000AVJIBQAX"}}, {"Skill_Name__c":"Verbal Commu
15:39:50:017	STATEMENT_EX...	[8]
15:39:50:017	HEAP_ALLOCATE	[8]Bytes:4
15:39:50:017	VARIABLE_ASSI...	[8] skillNames [] 0x630c57df
15:39:50:017	HEAP_ALLOCATE	[9]Bytes:5
15:39:50:017	HEAP_ALLOCATE	[9]Bytes:12
15:39:50:017	VARIABLE_SCO...	[9] s Skill__c true false
15:39:50:017	VARIABLE_ASSI...	[9] s {"Skill_Name__c":"Verbal Communication (1 more) ...","Id":"a08gL00000AVJIBQAX"} 0x59d7a547
15:39:50:017	STATEMENT_EX...	[9]
15:39:50:017	STATEMENT_EX...	[10]
15:39:50:017	HEAP_ALLOCATE	[EXTERNAL] Bytes:4
15:39:50:017	HEAP_ALLOCATE	[9]Bytes:5
15:39:50:017	HEAP_ALLOCATE	[9]Bytes:12
15:39:50:017	VARIABLE_SCO...	[9] s Skill__c true false
15:39:50:017	VARIABLE_ASSI...	[9] s {"Skill_Name__c":"Verbal Communication (1 more) ...","Id":"a08gL00000AVWYvQAP"} 0x6f1e7a41
15:39:50:017	STATEMENT_EX...	[9]
15:39:50:017	STATEMENT_EX...	[10]
15:39:50:017	HEAP_ALLOCATE	[EXTERNAL] Bytes:4
15:39:50:017	HEAP_ALLOCATE	[9]Bytes:5
15:39:50:017	VARIABLE_ASSI...	[9] s null

2. **Search Candidates, Skills, Jobs at Once**
(Use the SOSL example above).

5. Collections (List, Set, Map)

- **List** → Store multiple training enrollments.
- **Set** → Prevent duplicate skill assignments.
- **Map** → Map candidate IDs to their completed courses.

Apex Collections

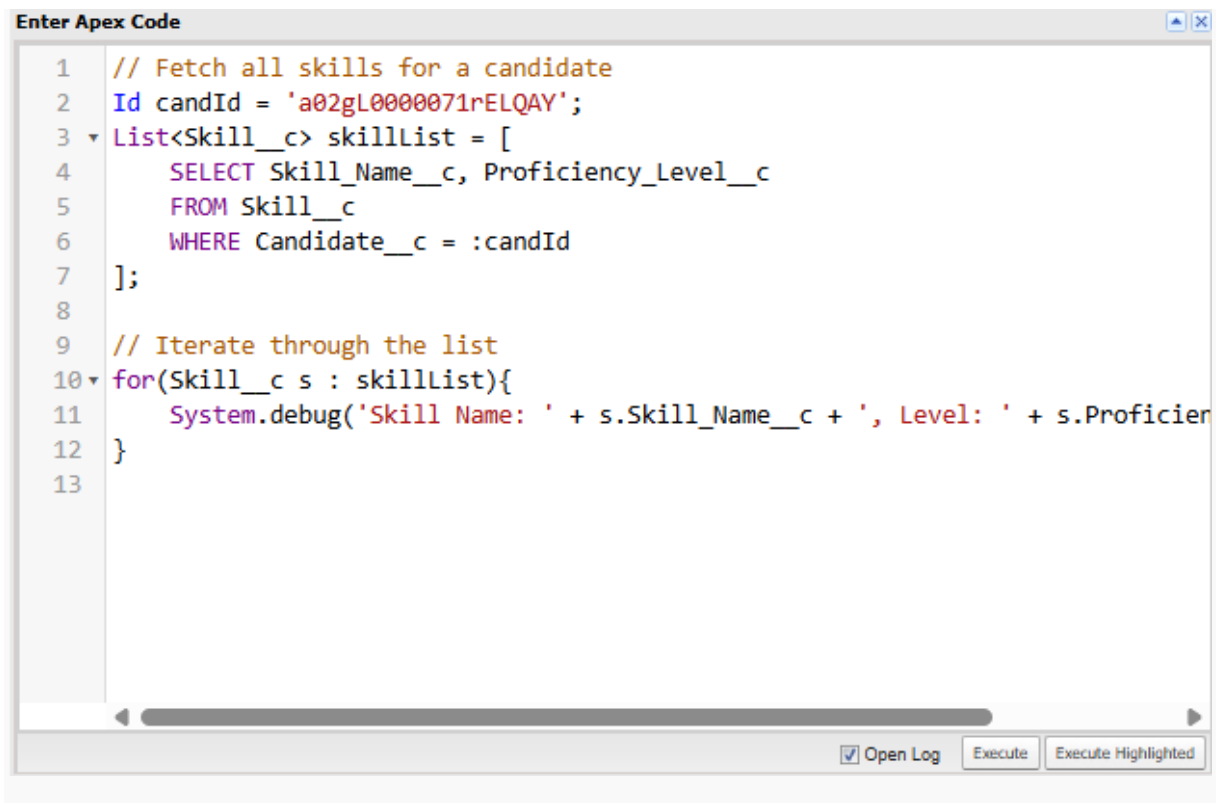
Collections are **variables that store multiple values** in a single variable. Apex supports three types:

Type	Description	Example Use in Your Project
List	Ordered collection (like array)	Store all Skills of a Candidate
Set	Unordered collection with unique values	Keep unique Skill Names
Map	Key-value pairs	Map Candidate Id → Candidate record

1 Lists

Example: List of Skills for a Candidate

```
// Fetch all skills for a candidate Id candId = 'PUT_CANDIDATE_ID_HERE';
List<Skill__c> skillList = [ SELECT Skill_Name__c, Proficiency_Level__c
FROM Skill__c WHERE Candidate__c = :candId ]; // Iterate through the list
for(Skill__c s : skillList){ System.debug('Skill Name: ' + s.Skill_Name__c
+ ', Level: ' + s.Proficiency_Level__c); }
```



```
1 // Fetch all skills for a candidate
2 Id candId = 'a02gL0000071rELQAY';
3 List<Skill__c> skillList = [
4     SELECT Skill_Name__c, Proficiency_Level__c
5     FROM Skill__c
6     WHERE Candidate__c = :candId
7 ];
8
9 // Iterate through the list
10 for(Skill__c s : skillList){
11     System.debug('Skill Name: ' + s.Skill_Name__c + ', Level: ' + s.Proficiency_Level__c);
12 }
13
```

Open Log Execute Execute Highlighted

Use Case: Display all skills of a candidate

15:44:28:017	HEAP_ALLOCATE	[11] Bytes:46
15:44:28:017	USER_DEBUG	[11] DEBUG Skill Name: Verbal Communications, Level: null

2 Sets

Example: Unique Skill Names

```
List<Skill__c> skills = [ SELECT Skill_Name__c FROM Skill__c WHERE
Candidate__c = :candId ]; Set<String> uniqueSkills = new Set<String>();
for(Skill__c s : skills){ uniqueSkills.add(s.Skill_Name__c); }
System.debug('Unique Skills: ' + uniqueSkills);
```

Enter Apex Code

1

2

3

4

5

6

7

8

9

10

11

12

13

List<Skill__c> skills = [
 SELECT Skill_Name__c
 FROM Skill__c
 WHERE Candidate__c = : 'a02gL0000071rELQAY'
];

Set<String> uniqueSkills = new Set<String>();
for(Skill__c s : skills){
 uniqueSkills.add(s.Skill_Name__c);
}

System.debug('Unique Skills: ' + uniqueSkills);

☒ Open Log

Execute

Execute Highlighted

orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexC...		
File Edit Debug Test Workspace Help < >		
Log executeAnonymous @9/21/2025, 3:39:50 PM Log executeAnonymous @9/21/2025, 3:44:28 PM Log executeAnonymous @9/21/2025, 3:47:05 PM		
Execution Log		
Timestamp	Event	Details
15:47:05:019	STATEMENT_EX...	[8]]
15:47:05:019	STATEMENT_EX...	[9]]
15:47:05:019	HEAP_ALLOCATE	[EXTERNAL]]Bytes:4
15:47:05:019	HEAP_ALLOCATE	[8]]Bytes:5
15:47:05:019	HEAP_ALLOCATE	[8]]Bytes:12
15:47:05:019	VARIABLE_SCO...	[8]]s Skill__c true false
15:47:05:019	VARIABLE_ASSI...	[8]]s {"Skill_Name__c":"Verbal Communication (1 more) ...","Id":"a0BgL00000AVWYvQAP"} 0x3779341c
15:47:05:019	STATEMENT_EX...	[8]]
15:47:05:019	STATEMENT_EX...	[9]]
15:47:05:019	HEAP_ALLOCATE	[EXTERNAL]]Bytes:4
15:47:05:019	HEAP_ALLOCATE	[8]]Bytes:5
15:47:05:019	VARIABLE_ASSI...	[8]]s null
15:47:05:019	STATEMENT_EX...	[12]]
15:47:05:019	HEAP_ALLOCATE	[12]]Bytes:15
15:47:05:019	HEAP_ALLOCATE	[12]]Bytes:23
15:47:05:019	HEAP_ALLOCATE	[12]]Bytes:38
15:47:05:019	USER_DEBUG	[12]]DEBUG Unique Skills: {Verbal Communications}
15:47:05:019	CUMULATIVE_L...	
15:47:05:019	LIMIT_USAGE_...	(default)
15:47:05:000	LIMIT_USAGE_...	Number of SOQL queries: 1 out of 100
15:47:05:000	LIMIT_USAGE_...	Number of query rows: 2 out of 50000
15:47:05:000	LIMIT_USAGE_...	Number of SOSL queries: 0 out of 20
15:47:05:000	LIMIT_USAGE_...	Number of DML statements: 0 out of 150
15:47:05:000	LIMIT_USAGE_...	Number of Publish Immediate DML: 0 out of 150
15:47:05:000	LIMIT_USAGE_...	Number of DML rows: 0 out of 10000
15:47:05:000	LIMIT_USAGE_...	Maximum CPU time: 0 out of 10000
15:47:05:000	LIMIT_USAGE_...	Maximum heap size: 0 out of 6000000
15:47:05:000	LIMIT_USAGE_...	Number of callouts: 0 out of 100
15:47:05:000	LIMIT_USAGE_...	Number of Email Invocations: 0 out of 10
15:47:05:000	LIMIT_USAGE_...	Number of future calls: 0 out of 50
15:47:05:000	LIMIT_USAGE_...	Number of queueable jobs added to the queue: 0 out of 50
15:47:05:000	LIMIT_USAGE_...	Number of Mobile Apex push calls: 0 out of 10
15:47:05:000	LIMIT_USAGE_...	
15:47:05:019	CUMULATIVE_L...	

Use Case: Avoid duplicate skills when assigning new ones.

3 Maps

Example: Candidate Id → Candidate Record

```
// Fetch all candidates in a department List<Candidate__c> candList = [
SELECT Id, Email__c, Education__c FROM Candidate__c WHERE Education__c = 'B
Tech' ]; // Convert to map for easy lookup Map<Id, Candidate__c>
candidateMap = new Map<Id, Candidate__c>(candList); // Access a candidate
quickly by Id Id myCandId = 'PUT_CANDIDATE_ID_HERE'; Candidate__c
myCandidate = candidateMap.get(myCandId); System.debug('Candidate Email: '
+ myCandidate.Email__c);
```

Enter Apex Code

```
1 // Fetch all candidates in a department
2 List<Candidate__c> candList = [
3     SELECT Id, Email__c, Education__c
4     FROM Candidate__c
5     WHERE Education__c = 'B Tech'
6 ];
7
8 // Convert to map for easy lookup
9 Map<Id, Candidate__c> candidateMap = new Map<Id, Candidate__c>(candList);
10
11 // Access a candidate quickly by Id
12 Id myCandId = 'a02gL0000071rELQAY';
13 Candidate__c myCandidate = candidateMap.get(myCandId);
14 System.debug('Candidate Email: ' + myCandidate.Email__c);
15
```

☒ Open Log

Execute

Execute Highlighted

Execution Log		
Timestamp	Event	Details
15:52:55:014	HEAP_ALLOCATE	[2]]Bytes:12
15:52:55:014	HEAP_ALLOCATE	[2]]Bytes:243
15:52:55:014	HEAP_ALLOCATE	[2]]Bytes:12
15:52:55:014	VARIABLE_ASSI...	[2]]candList[[{"RecordTypeId":"012gL000002UWHRQA4","Id":"a02gL0000071rELQAY","Education__c":"B Tech","Email...
15:52:55:014	STATEMENT_EX...	[9]]
15:52:55:014	HEAP_ALLOCATE	[9]]Bytes:4
15:52:55:014	HEAP_ALLOCATE	[9]]Bytes:20
15:52:55:014	HEAP_ALLOCATE	[9]]Bytes:4
15:52:55:014	HEAP_ALLOCATE	[9]]Bytes:20
15:52:55:014	HEAP_ALLOCATE	[9]]Bytes:4
15:52:55:014	VARIABLE_ASSI...	[9]]candidateMap[{"a02gL0000071rELQAY":{"RecordTypeId":"012gL000002UWHRQA4","Id":"a02gL0000071rELQAY","
15:52:55:014	STATEMENT_EX...	[12]]
15:52:55:014	HEAP_ALLOCATE	[12]]Bytes:18
15:52:55:014	HEAP_ALLOCATE	[12]]Bytes:4
15:52:55:015	VARIABLE_ASSI...	[12]]myCandId]"a02gL0000071rELQAY"
15:52:55:015	STATEMENT_EX...	[13]]
15:52:55:015	VARIABLE_ASSI...	[13]]myCandidate[{"RecordTypeId":"012gL000002UWHRQA4","Id":"a02gL0000071rELQAY","Education__c":"B Tech","E
15:52:55:015	STATEMENT_EX...	[14]]
15:52:55:015	HEAP_ALLOCATE	[14]]Bytes:17
15:52:55:015	HEAP_ALLOCATE	[14]]Bytes:47
15:52:55:015	USER_DEBUG	[14]]DEBUG Candidate Email: rupasreethulasiraman@gmail.com
15:52:55:015	CUMULATIVE_L...	
15:52:55:015	LIMIT_USAGE_...	(default)]
15:52:55:000	LIMIT_USAGE_...	Number of SQL queries: 1 out of 100
15:52:55:000	LIMIT_USAGE_...	Number of query rows: 2 out of 50000
15:52:55:000	LIMIT_USAGE_...	Number of SOSL queries: 0 out of 20
15:52:55:000	LIMIT_USAGE_...	Number of DML statements: 0 out of 150
15:52:55:000	LIMIT_USAGE_...	Number of Publish Immediate DML: 0 out of 150
15:52:55:000	LIMIT_USAGE_...	Number of DML rows: 0 out of 10000
15:52:55:000	LIMIT_USAGE_...	Maximum CPU time: 0 out of 10000

Use Case: When matching candidates to jobs based on skill sets, you can quickly look up a candidate without looping through a list.

4 Combine Collections

Example: Filter Jobs Based on Candidate Skills

```
Id candId = 'PUT_CANDIDATE_ID_HERE'; // Step 1: Get candidate skills
List<Skill__c> skills = [ SELECT Skill_Name__c FROM Skill__c WHERE
Candidate__c = :candId ]; Set<String> skillSet = new Set<String>();
for(Skill__c s : skills){ skillSet.add(s.Skill_Name__c); } // Step 2: Fetch
Jobs where department matches any skill List<Job__c> jobs = [ SELECT Id,
Job_Title__c, Job_Department__c FROM Job__c WHERE Job_Department__c IN
:skillSet ]; System.debug('Matching Jobs: ' + jobs);
```

Enter Apex Code

```
1 Id candId = 'a02gL0000071rELQAY';
2
3 // Step 1: Get candidate skills
4 List<Skill__c> skills = [
5     SELECT Skill_Name__c
6     FROM Skill__c
7     WHERE Candidate__c = :candId
8 ];
9
10 Set<String> skillSet = new Set<String>();
11 for(Skill__c s : skills){
12     skillSet.add(s.Skill_Name__c);
13 }
14
15 // Step 2: Fetch Jobs where department matches any skill
16 List<Job__c> jobs = [
17     SELECT Id, Job_Title__c, Job_Department__c
18     FROM Job__c
19     WHERE Job_Department__c IN :skillSet
20 ];
21
22 System.debug('Matching Jobs: ' + jobs);
23
```

Explanation:

- **Set** prevents duplicate skill names.
- **List** stores Jobs that match those skills.

Edit Debug Test Workspace Help Log executeAnonymous @9/21/2025, 3:49:05 PM Log executeAnonymous @9/21/2025, 3:52:55 PM Log executeAnonymous @9/21/2025, 3:55:29 PM		
Execution Log		
Timestamp	Event	Details
55:29:015	HEAP_ALLOCATE	[11]Bytes:5
55:29:015	HEAP_ALLOCATE	[11]Bytes:12
55:29:015	VARIABLE_SCO...	[11]s Skill__c true false
55:29:015	VARIABLE_ASSI...	[11]s {"Skill_Name__c":"Verbal Communication (1 more) ...","Id":"a08gL00000AVWYvQAP"} 0x3de0f51c
55:29:015	STATEMENT_EX...	[11]
55:29:015	STATEMENT_EX...	[12]
55:29:015	HEAP_ALLOCATE	[EXTERNAL]Bytes:4
55:29:015	HEAP_ALLOCATE	[11]Bytes:5
55:29:015	VARIABLE_ASSI...	[11]s null
55:29:015	STATEMENT_EX...	[16]
55:29:015	HEAP_ALLOCATE	[16]Bytes:89
55:29:015	HEAP_ALLOCATE	[16]Bytes:4
55:29:015	HEAP_ALLOCATE	[16]Bytes:8
55:29:015	HEAP_ALLOCATE	[16]Bytes:8
55:29:016	SOQL_EXECUTE...	[16] Aggregations:0 SELECT Id, Job_Title__c, Job_Department__c FROM Job__c WHERE Job_Department__c = :tmpVar1
55:29:027	SOQL_EXECUTE...	[16]Rows:0
55:29:028	HEAP_ALLOCATE	[16]Bytes:4
55:29:028	HEAP_ALLOCATE	[16]Bytes:0
55:29:028	HEAP_ALLOCATE	[16]Bytes:4
55:29:028	VARIABLE_ASSI...	[16] jobs [] 0x6c59fc01
55:29:028	STATEMENT_EX...	[22]
55:29:028	HEAP_ALLOCATE	[22]Bytes:15
55:29:028	HEAP_ALLOCATE	[22]Bytes:2
55:29:028	HEAP_ALLOCATE	[22]Bytes:17
55:29:028	USER_DEBUG	[22]DEBUG Matching Jobs: ()
55:29:028	CUMULATIVE_L...	
55:29:028	LIMIT_USAGE_...	(default)
55:29:000	LIMIT_USAGE_...	Number of SOQL queries: 2 out of 100
55:29:000	LIMIT_USAGE_...	Number of query rows: 2 out of 50000
55:29:000	LIMIT_USAGE_...	Number of SOSL queries: 0 out of 20
55:29:000	LIMIT_USAGE_...	Number of DML statements: 0 out of 150
55:29:000	LIMIT_USAGE_...	Number of Publish Immediate DML: 0 out of 150
55:29:000	LIMIT_USAGE_...	Number of DML rows: 0 out of 10000
55:29:000	LIMIT_USAGE_...	Maximum CPU time: 0 out of 10000

This Frame
☐ Executable
☐ Debug Only
☐ Filter
Click here to filter the log

Filters job based on skills, as there is no job here shows empty set

5 Test Collections in Developer Console

1. Open **Developer Console** → **Execute Anonymous**
2. Paste the above example for **filtering jobs by candidate skills**
3. Click **Execute**
4. Check **Logs** → **USER_DEBUG** for outputs

This completes **Collections (List, Set, Map)** and how to use them practically in your **Skill & Employment Portal**.

6. Control Statements

- `IF-ELSE` → Validate candidate eligibility.
- `FOR loops` → Process multiple enrollments/applications in bulk.

Control statements help you **decide the flow of code**—like loops, conditionals, and branching. They are essential when processing Candidates, Skills, Trainings, or Jobs.

1 IF-ELSE Statements

✓ **Example: Assign Skill Based on Education**

```
Enter Apex Code
1 // Step 1: Query a candidate first (replace email with an existing candidate)
2 Candidate__c cand = [
3     SELECT Id, Education__c
4     FROM Candidate__c
5     WHERE Email__c = 'nirup2028@gmail.com'
6     LIMIT 1
7 ];
8
9 // Step 2: Assign Skill based on Education
10 Skill__c skill = new Skill__c(Candidate__c = cand.Id);
11
12 if(cand.Education__c == 'B Tech') {
13     skill.Skill_Name__c = 'Programming';
14 } else if(cand.Education__c == 'MCA') {
15     skill.Skill_Name__c = 'Advanced Programming';
16 } else {
17     skill.Skill_Name__c = 'General Skill';
18 }
19
20 // Step 3: Insert Skill
21 insert skill;
22 System.debug('Skill assigned: ' + skill.Skill_Name__c);
23
```

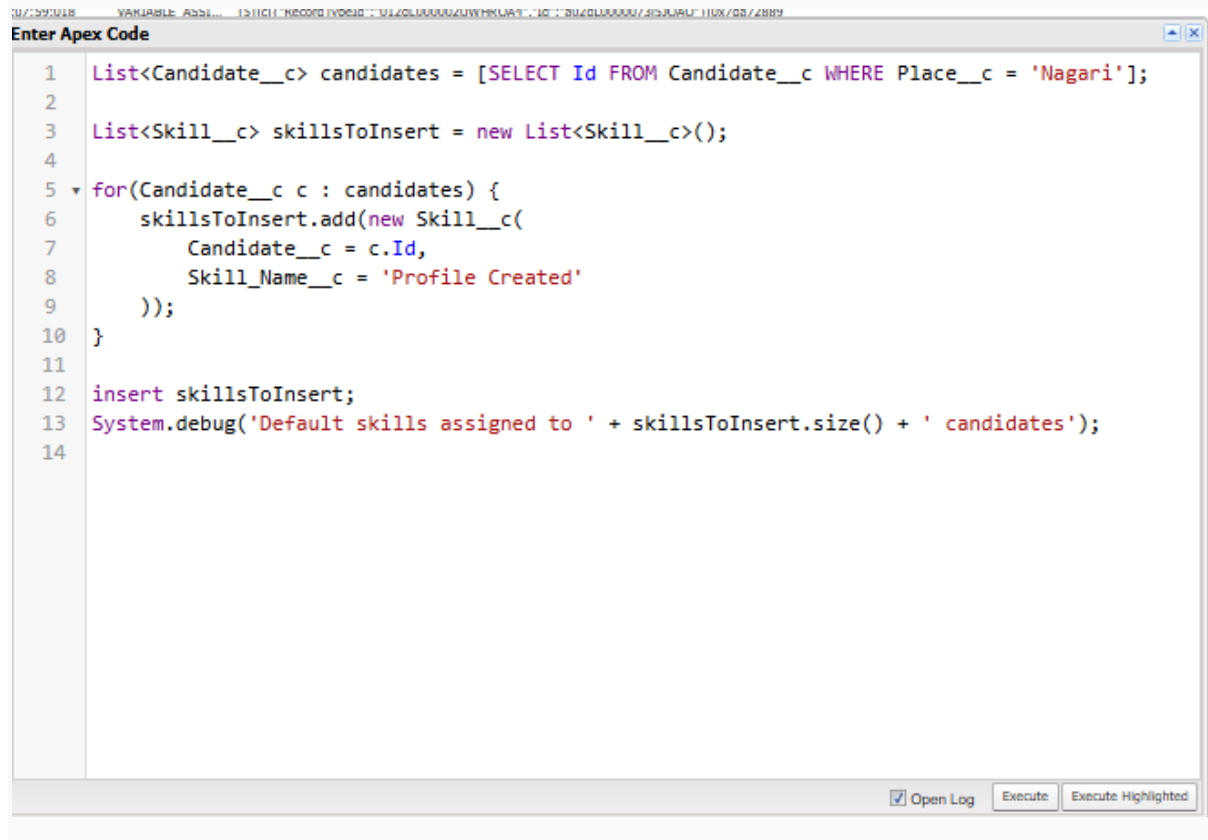
Use Case: Auto-assign skills based on candidate education.

Timestamp	Event	Details
16:04:16:023	HEAP_ALLOCATE	[10] Bytes:4
16:04:16:023	VARIABLE_ASSIGN	[10] this.Candidate__c "a02gL0000073I5JQAQ" 0x386415ff
16:04:16:023	VARIABLE_ASSIGN	[10] skill {"Candidate__c":"a02gL0000073I5JQAQ"} 0x386415ff
16:04:16:023	HEAP_ALLOCATE	[12] Bytes:6
16:04:16:023	STATEMENT_EXECUTE	[12]
16:04:16:023	STATEMENT_EXECUTE	[13]
16:04:16:023	HEAP_ALLOCATE	[13] Bytes:11
16:04:16:023	VARIABLE_ASSIGN	[13] this.Skill_Name__c "Programming" 0x386415ff
16:04:16:023	STATEMENT_EXECUTE	[21]
16:04:16:023	HEAP_ALLOCATE	[21] Bytes:8
16:04:16:023	DML_BEGIN	[21] Op:Insert Type:Skill__c Rows:1
16:04:16:023	HEAP_ALLOCATE	[EXTERNAL] Bytes:8
16:04:16:085	DML_END	[21]
16:04:16:086	STATEMENT_EXECUTE	[22]
16:04:16:086	HEAP_ALLOCATE	[22] Bytes:16
16:04:16:086	HEAP_ALLOCATE	[22] Bytes:27
16:04:16:086	USER_DEBUG	[22] DEBUG Skill assigned: Programming
16:04:16:123	CUMULATIVE_LIMIT_USAGE	
16:04:16:123	LIMIT_USAGE	(default)
16:04:16:000	LIMIT_USAGE	Number of SOQL queries: 1 out of 100
16:04:16:000	LIMIT_USAGE	Number of queries executed: 1 out of 50000

2 FOR Loops

Example: Assign Default Skill to Multiple Candidates

```
List<Candidate__c> candidates = [SELECT Id FROM Candidate__c WHERE Place__c = 'Nagari']; List<Skill__c> skillsToInsert = new List<Skill__c>(); for(Candidate__c c : candidates) { skillsToInsert.add(new Skill__c( Candidate__c = c.Id, Skill_Name__c = 'Profile Created' )); } insert skillsToInsert; System.debug('Default skills assigned to ' + skillsToInsert.size() + ' candidates');
```



Execution Log		
Timestamp	Event	Details
16:07:59:018	HEAP_ALLOCATE	[5] Bytes:5
16:07:59:018	HEAP_ALLOCATE	[5] Bytes:12
16:07:59:018	VARIABLE_SCO...	[5] c Candidate__c true false
16:07:59:018	VARIABLE_ASSI...	[5] c {"RecordTypeId":"012gL000002UWHRQA4","Id":"a02gL0000073i5JQAQ"} 0x7da72889
16:07:59:018	STATEMENT_EX...	[5]
16:07:59:018	STATEMENT_EX...	[6]
16:07:59:018	HEAP_ALLOCATE	[6] Bytes:4
16:07:59:018	VARIABLE_ASSI...	[6] this.Candidate__c "a02gL0000073i5JQAQ" 0x250981c9
16:07:59:018	VARIABLE_ASSI...	[6] this.Skill_Name__c "Profile Created" 0x250981c9
16:07:59:018	HEAP_ALLOCATE	[EXTERNAL] Bytes:4
16:07:59:018	HEAP_ALLOCATE	[5] Bytes:5
16:07:59:018	VARIABLE_ASSI...	[5] c null
16:07:59:018	STATEMENT_EX...	[12]
16:07:59:019	DML_BEGIN	[12] Op:Insert Type:Skill__c Rows:2
16:07:59:019	HEAP_ALLOCATE	[EXTERNAL] Bytes:12
16:07:59:076	DML_END	[12]
16:07:59:077	HEAP_ALLOCATE	[EXTERNAL] Bytes:12
16:07:59:077	STATEMENT_EX...	[13]
16:07:59:077	HEAP_ALLOCATE	[13] Bytes:27
16:07:59:077	HEAP_ALLOCATE	[13] Bytes:1
16:07:59:077	HEAP_ALLOCATE	[13] Bytes:11
16:07:59:077	HEAP_ALLOCATE	[13] Bytes:39
16:07:59:077	USER_DEBUG	[13] (DEBUG) Default skills assigned to 2 candidates
16:07:59:118	CUMULATIVE_L...	
16:07:59:118	LIMIT_USAGE_...	(default)
16:07:59:000	LIMIT_USAGE_...	Number of SOQL queries: 1 out of 100
16:07:59:000	LIMIT_USAGE_...	Number of query rows: 2 out of 50000
16:07:59:000	LIMIT_USAGE_...	Number of SOSL queries: 0 out of 20
16:07:59:000	LIMIT_USAGE_...	Number of DML statements: 1 out of 150
16:07:59:000	LIMIT_USAGE_...	Number of Publish Immediate DML: 0 out of 150

3 WHILE Loops

Example: Process Jobs Until Threshold

```
List<Job__c> jobs = [SELECT Id, Job_Title__c FROM Job__c LIMIT 5]; Integer
i = 0; while(i < jobs.size()) { System.debug('Processing Job: ' +
jobs[i].Job_Title__c); i++; }
```

nter Apex Code

```

1 List<Job__c> jobs = [SELECT Id, Job_Title__c FROM Job__c LIMIT 5];
2 Integer i = 0;
3
4 while(i < jobs.size()) {
5     System.debug('Processing Job: ' + jobs[i].Job_Title__c);
6     i++;
7 }
8

```

☒ Open Log
 Execute
 Execute Highlighted

File Edit Debug Test Workspace Help

25, 3:55:29 PM

Log executeAnonymous @9/21/2025, 4:04:16 PM

Log executeAnonymous @9/21/2025, 4:07:59 PM

Log executeAnonymous @9/21/2025, 4:10:18 PM

Execution Log

Timestamp	Event	Details
16:10:18:015	USER_DEBUG	[5] [DEBUG] Processing Job: Backend Developer Test
16:10:18:015	STATEMENT_EX...	[6]
16:10:18:015	HEAP_ALLOCATE	[6] Bytes: 4
16:10:18:015	VARIABLE_ASSI...	[6] 3
16:10:18:015	STATEMENT_EX...	[4]
16:10:18:015	STATEMENT_EX...	[5]
16:10:18:015	HEAP_ALLOCATE	[5] Bytes: 16
16:10:18:015	HEAP_ALLOCATE	[5] Bytes: 38
16:10:18:015	USER_DEBUG	[5] [DEBUG] Processing Job: Backend Developer Test
16:10:18:015	STATEMENT_EX...	[6]
16:10:18:015	HEAP_ALLOCATE	[6] Bytes: 4
16:10:18:015	VARIABLE_ASSI...	[6] 4
16:10:18:015	STATEMENT_EX...	[4]
16:10:18:015	STATEMENT_EX...	[5]
16:10:18:015	HEAP_ALLOCATE	[5] Bytes: 12
16:10:18:015	HEAP_ALLOCATE	[5] Bytes: 30
16:10:18:015	USER_DEBUG	[5] [DEBUG] Processing Job: null
16:10:18:015	STATEMENT_EX...	[6]
16:10:18:015	HEAP_ALLOCATE	[6] Bytes: 4
16:10:18:015	VARIABLE_ASSI...	[6] 5
16:10:18:016	CUMULATIVE_L...	
16:10:18:016	LIMIT_USAGE...	(default)
16:10:18:000	LIMIT_USAGE...	Number of SOQL queries: 1 out of 100
16:10:18:000	LIMIT_USAGE...	Number of query rows: 5 out of 50000
16:10:18:000	LIMIT_USAGE...	Number of SOQL queries: 0 out of 20
16:10:18:000	LIMIT_USAGE...	Number of DML statements: 0 out of 150

4 SWITCH Statements (Apex 50+)

Example: Assign Skill Category Based on Department

```

String dept = 'IT'; String category; switch on dept { when 'IT' { category
= 'Technical'; } when 'HR' { category = 'Management'; } when 'Sales' {
category = 'Business'; } when else { category = 'General'; } }
System.debug('Skill Category: ' + category);

```

Enter Apex Code

```
1 String dept = 'IT';
2 String category;
3
4 switch on dept {
5     when 'IT' {
6         category = 'Technical';
7     }
8     when 'HR' {
9         category = 'Management';
10    }
11    when 'Sales' {
12        category = 'Business';
13    }
14    when else {
15        category = 'General';
16    }
17 }
18
19 System.debug('Skill Category: ' + category);
20
```

☒ Open Log Execute Execute Highlighted

18:20:28:000 C:\P21_03\IDE_... Number of Carous: 0 out of 100

File Edit Debug Test Workspace Help < >

5, 4:04:16 PM Log executeAnonymous @9/21/2025, 4:07:59 PM Log executeAnonymous @9/21/2025, 4:10:18 PM Log ex...

Execution Log

Timestamp	Event	Details
16:13:09:001	USER_INFO	[EXTERNAL]]005gL000005Dp8H rupasree287375@ageantforce.com (GMT-07:00) Pacific Daylight Time (America/Los...
16:13:09:001	EXECUTION_ST...	
16:13:09:001	CODE_UNIT_ST...	[EXTERNAL]]execute_anonymous_apex
16:13:09:001	VARIABLE_SCO...	[2]]category String false false
16:13:09:001	VARIABLE_SCO...	[1]]dept String false false
16:13:09:001	HEAP_ALLOCATE	[95]]Bytes:3
16:13:09:002	HEAP_ALLOCATE	[100]]Bytes:152
16:13:09:002	HEAP_ALLOCATE	[417]]Bytes:408
16:13:09:002	HEAP_ALLOCATE	[430]]Bytes:408
16:13:09:002	HEAP_ALLOCATE	[317]]Bytes:6
16:13:09:002	HEAP_ALLOCATE	[EXTERNAL]]Bytes:25
16:13:09:002	STATEMENT_EX...	[1]]
16:13:09:002	STATEMENT_EX...	[1]]
16:13:09:002	HEAP_ALLOCATE	[1]]Bytes:2
16:13:09:002	VARIABLE_ASSI...	[1]]dept "IT"
16:13:09:002	STATEMENT_EX...	[2]]
16:13:09:002	VARIABLE_ASSI...	[2]]category null
16:13:09:002	STATEMENT_EX...	[4]]
16:13:09:002	STATEMENT_EX...	[5]]
16:13:09:002	STATEMENT_EX...	[6]]
16:13:09:002	HEAP_ALLOCATE	[6]]Bytes:9
16:13:09:002	VARIABLE_ASSI...	[6]]category "Technical"
16:13:09:002	STATEMENT_EX...	[19]]
16:13:09:002	HEAP_ALLOCATE	[19]]Bytes:16
16:13:09:002	HEAP_ALLOCATE	[19]]Bytes:25
16:13:09:002	HEAP_ALLOCATE	[68]]Bytes:5
16:13:09:002	HEAP_ALLOCATE	[74]]Bytes:5
16:13:09:002	HEAP_ALLOCATE	[82]]Bytes:7
16:13:09:002	USER_DEBUG	[19]]DEBUG Skill Category: Technical
16:13:09:002	CUMULATIVE_I...	
16:13:09:002	LIMIT_USAGE_...	(default)]
16:13:09:000	LIMIT_USAGE_...	Number of SOQL queries: 0 out of 100
16:13:09:000	LIMIT_USAGE_...	Number of query rows: 0 out of 50000
16:13:09:000	LIMIT_USAGE_...	Number of SOSL queries: 0 out of 20

5 Practical usecases of our Project

- Use **IF-ELSE** to validate inputs (e.g., check candidate email, contact number).
- Use **FOR loops** to bulk process multiple candidates, jobs, or skills.
- Use **SWITCH** to categorize jobs, skills, or training programs.
- Avoid **SOQL inside loops**—always query first, then loop through results.

Test in Developer Console

```
List<Candidate__c> candidates = [SELECT Id, Education__c FROM Candidate__c LIMIT 3]; for(Candidate__c c : candidates) { if(c.Education__c == 'B Tech') { System.debug('Assign Programming Skill to: ' + c.Id); } else { System.debug('Assign General Skill to: ' + c.Id); } }
```

- Open **Execute Anonymous** → **Run** → **Check Logs** → **USER_DEBUG**
 - You will see which candidate gets which skill.
-

7. Batch Apex

Batch Apex allows you to process **thousands of records asynchronously** in batches of 200 by default.

A batch class implements the `Database.Batchable<SObject>` interface and has **3 main methods**:

Method	Purpose
<code>start</code>	Collects the records to process (returns a QueryLocator or iterable)
<code>execute</code>	Processes each batch of records
<code>finish</code>	Executes code after all batches are processed

1 Use Case for our Project

Scenario: Assign a default skill to all candidates who completed a training program.

- Object: `Candidate__c`
- Related Object: `Training__c`
- Create `Skill__c` records for each candidate after training completion.

2 Example Batch Apex Class

```
global class AssignSkillBatch implements Database.Batchable<SObject> { //
Step 1: start() - Fetch all completed trainings global
Database.QueryLocator start(Database.BatchableContext BC) { return
Database.getQueryLocator([ SELECT Id, Candidate__c, Training_Name__c FROM
Training__c WHERE Training_Status__c = 'Completed' ]); } // Step 2:
execute() - Process each batch of records global void
execute(Database.BatchableContext BC, List<Training__c> scope) {
List<Skill__c> skillsToInsert = new List<Skill__c>(); for(Training__c tr :
scope) { skillsToInsert.add(new Skill__c( Candidate__c = tr.Candidate__c,
Skill_Name__c = tr.Training_Name__c )); } if(!skillsToInsert.isEmpty()) {
insert skillsToInsert; } } // Step 3: finish() - Optional: Send
notification or log global void finish(Database.BatchableContext BC) {
System.debug('Batch Apex completed: Skills assigned successfully!'); } }
```

 **SETUP**
Apex Classes

Apex Class

Help for this Page ?

AssignSkillBatch

Apex Class Detail

EditDeleteDownloadSecurityShow Dependencies

Name	AssignSkillBatch	Status	Active
Namespace Prefix		Code Coverage	0% (0/11)
Created By	T. Rupasree , 9/21/2025, 3:50 AM	Last Modified By	T. Rupasree , 9/21/2025, 3:50 AM

Class Body

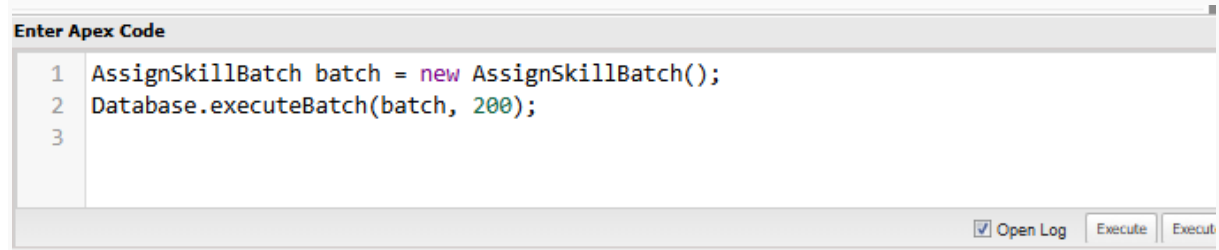
Class SummaryVersion SettingsTrace Flags

```
1  global class AssignSkillBatch implements Database.Batchable<SObject> {
2
3      global Database.QueryLocator start(Database.BatchableContext BC) {
4          return Database.getQueryLocator([
5              SELECT Id, Candidate__c, Training_Name__c
6              FROM Training__c
7              WHERE Training_Status__c = 'Completed'
8          ]);
9      }
10
11     global void execute(Database.BatchableContext BC, List<Training__c> scope) {
12         List<Skill__c> skillsToInsert = new List<Skill__c>();
13
14         for(Training__c tr : scope) {
15             skillsToInsert.add(new Skill__c(
16                 Candidate__c = tr.Candidate__c,
17                 Skill_Name__c = tr.Training_Name__c
18             ));
19         }
20
21         if(!skillsToInsert.isEmpty()) {
22             insert skillsToInsert;
23         }
24     }
25
26     global void finish(Database.BatchableContext BC) {
27         System.debug('Batch Apex completed: Skills assigned successfully!');
28     }
29 }
```

3 How to Run Batch Apex

Option 1: Execute Anonymous

```
// Run batch with batch size of 200 AssignSkillBatch batch = new  
AssignSkillBatch(); Database.executeBatch(batch, 200);
```

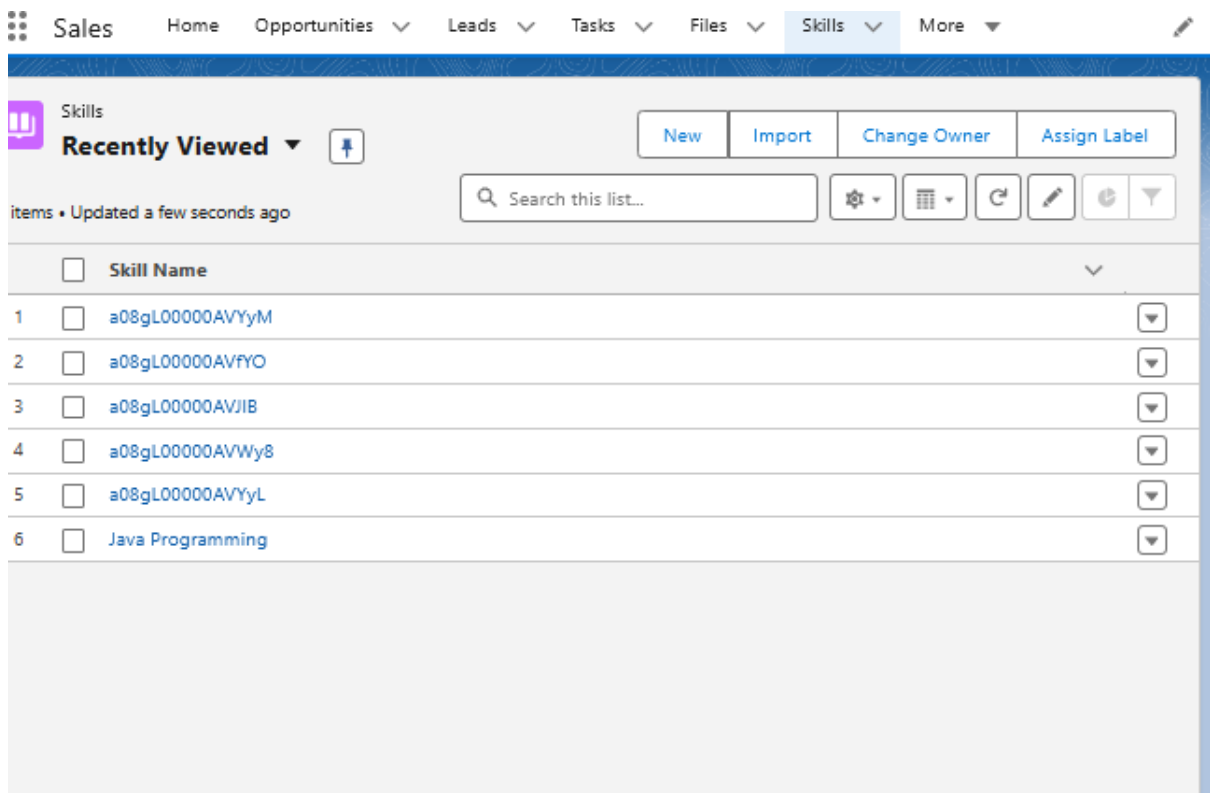


Option 2: Schedule Batch (optional)

```
String cronExp = '0 0 23 * * ?'; // Run daily at 11 PM  
System.schedule('Daily Skill Assignment', cronExp, new AssignSkillBatch());
```

4 How It Works

1. `start()` → Fetch all `Training__c` records with status **Completed**
2. `execute()` → For each batch of 200 trainings, create corresponding `Skill__c` records
3. `finish()` → Logs or sends notification when batch is complete
 - Use case: Bulk update **“Certified” status** for thousands of candidates after training completion.
 - Supports **Skill Development** at scale.



8. Queueable Apex

- Use case: Match skills to jobs when a new **job posting** is created.
- Asynchronous processing improves performance.

Queueable Apex is like a **lightweight version of Batch Apex**:

- Can be run **asynchronously**
- Can be **chained** (one job starts another)
- Supports **complex objects, SOjects, and collections**

1 Use Case in Your Project

Scenario: Automatically assign a skill to a candidate **right after a training is marked complete**, without waiting for a batch job.

- Object: `Candidate__c`

- Related Object: `Training__c`
- Create `Skill__c` record **asynchronously** after training completion.

2 Queueable Apex Class Example

```
public class AssignSkillQueueable implements Queueable { private Id
trainingId; // Constructor public AssignSkillQueueable(Id trId) {
this.trainingId = trId; } public void execute(QueueableContext context) {
// Fetch the Training record Training__c tr = [ SELECT Id, Candidate__c,
Training_Name__c FROM Training__c WHERE Id = :trainingId LIMIT 1 ]; //
Create the Skill record Skill__c skill = new Skill__c( Candidate__c =
tr.Candidate__c, Skill_Name__c = tr.Training_Name__c ); insert skill;
System.debug('Skill assigned asynchronously: ' + skill.Skill_Name__c); } }
```


Apex Classes

Apex Class Detail
Edit
Delete
Download
Security
Show Dependencies

Name	AssignSkillQueueable	Status	Active
Namespace Prefix		Code Coverage	0% (0/9)
Created By	T. Rupasree , 9/21/2025, 4:11 AM	Last Modified By	T. Rupasree , 9/21/2025, 4:11 AM

Class Body
Class Summary
Version Settings
Trace Flags

```

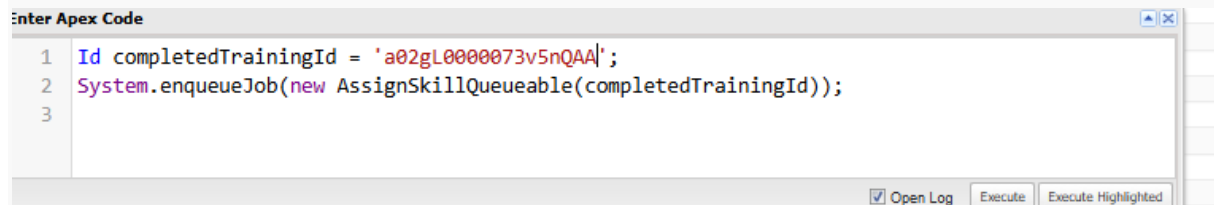
1 public class AssignSkillQueueable implements Queueable {
2
3     private Id trainingId;
4
5     // Constructor
6     public AssignSkillQueueable(Id trId) {
7         this.trainingId = trId;
8     }
9
10    public void execute(QueueableContext context) {
11
12        // Fetch the Training record
13        Training__c tr = [
14            SELECT Id, Candidate__c, Training_Name__c
15            FROM Training__c
16            WHERE Id = :trainingId
17            LIMIT 1
18        ];
19
20        // Create the Skill record
21        Skill__c skill = new Skill__c(
22            Candidate__c = tr.Candidate__c,
23            Skill_Name__c = tr.Training_Name__c
24        );
25
26        insert skill;
27        System.debug('Skill assigned asynchronously: ' + skill.Skill_Name__c);
28    }
29 }
```

Edit
Delete
Download
Security
Show Dependencies

3 How to Run Queueable Apex

Option 1: Execute Anonymous (Immediate)

```
Id completedTrainingId = 'PUT_TRAINING_RECORD_ID_HERE';
System.enqueueJob(new AssignSkillQueueable(completedTrainingId));
```



Option 2: Chain Queueable Jobs

```
// Inside execute(), you can enqueue another job if needed
System.enqueueJob(new AnotherQueueableJob(nextId));
```

4 Practical Implementation in our Project

1. Create the class **AssignSkillQueueable** in **Apex Classes**
2. After a training is marked **Completed**, run this queueable:
 - Either manually via Developer Console
 - Or trigger it automatically via **after update trigger on Training__c**

Example Trigger:

```
trigger TrainingTrigger on Training__c (after update) { for(Training__c tr
: Trigger.new) { Training__c oldTr = Trigger.oldMap.get(tr.Id); // Only
enqueue when status changes to Completed if(tr.Training_Status__c ==
'Completed' && oldTr.Training_Status__c != 'Completed') {
System.enqueueJob(new AssignSkillQueueable(tr.Id)); } } }
```

orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexC		
File Edit Debug Test Workspace Help < >		
Log executeAnonymous @9/21/2025, 4:13:09 PM Log executeAnonymous @9/21/2025, 4:21:10 PM Log executeAnonymous @9/21/2025, 4:23:08		
Execution Log		
Timestamp	Event	Details
16:43:49:001	STATEMENT_EX...	[1]
16:43:49:001	STATEMENT_EX...	[1]
16:43:49:001	HEAP_ALLOCATE	[1]Bytes:18
16:43:49:001	HEAP_ALLOCATE	[1]Bytes:4
16:43:49:001	VARIABLE_ASSI...	[1] completedTrainingId "a02gL0000073v5nQAA"
16:43:49:001	STATEMENT_EX...	[2]
16:43:49:009	HEAP_ALLOCATE	[2]Bytes:4
16:43:49:009	HEAP_ALLOCATE	[2]Bytes:2
16:43:49:009	METHOD_ENTRY	[1] 01pgL000005hQij AssignSkillQueueable.AssignSkillQueueable()
16:43:49:009	STATEMENT_EX...	[1]
16:43:49:009	STATEMENT_EX...	[1]
16:43:49:009	METHOD_EXIT	[1] AssignSkillQueueable
16:43:49:009	HEAP_ALLOCATE	[2]Bytes:8
16:43:49:009	HEAP_ALLOCATE	[68]Bytes:5
16:43:49:009	HEAP_ALLOCATE	[74]Bytes:5
16:43:49:009	HEAP_ALLOCATE	[82]Bytes:7
16:43:49:009	SYSTEM_MODE...	false
16:43:49:009	HEAP_ALLOCATE	[2]Bytes:5
16:43:49:009	CONSTRUCTOR...	[2] 01pgL000005hQij <init>(Id) AssignSkillQueueable
16:43:49:009	VARIABLE_SCO...	[6] this AssignSkillQueueable true false
16:43:49:009	VARIABLE_ASSI...	[6] this {} 0x1b00bb53
16:43:49:009	VARIABLE_SCO...	[6] trId Id false false
16:43:49:009	VARIABLE_ASSI...	[6] trId "a02gL0000073v5nQAA"
16:43:49:009	STATEMENT_EX...	[1]
16:43:49:009	STATEMENT_EX...	[3]
16:43:49:009	HEAP_ALLOCATE	[7]Bytes:6
16:43:49:010	STATEMENT_EX...	[6]
16:43:49:010	STATEMENT_EX...	[7]
16:43:49:010	VARIABLE_ASSI...	[7] this.trainingId "a02gL0000073v5nQAA" 0x1b00bb53
16:43:49:010	CONSTRUCTOR...	[2] 01pgL000005hQij <init>(Id) AssignSkillQueueable

Benefits

- Lightweight compared to batch
- Runs **asynchronously** without blocking UI
- Can process **large SOBJEcts** safely
- Can be **chained** for multiple sequential operations

9. Scheduled Apex

Scheduled Apex allows you to schedule **any Apex class that implements the `Schedulable` interface**.

Key Points:

- Must implement the `Schedulable` **interface**
 - Has `execute(SchedulableContext sc)` method
 - Can schedule **daily, weekly, or monthly** using **cron expressions**
-

1 Use Case in our Project

Scenario: Automatically assign skills to all candidates **who completed training** every day at 11 PM.

- Object: `Training__c`
 - Related Object: `Candidate__c`
 - Action: Run the **AssignSkillBatch** class automatically
-

2 Example Scheduled Apex Class

```
global class ScheduleAssignSkill implements Schedulable { global void
execute(SchedulableContext sc) { // Run the batch to assign skills
AssignSkillBatch batch = new AssignSkillBatch();
Database.executeBatch(batch, 200); System.debug('Scheduled Batch Apex
executed successfully!'); } }
```

Apex Class

ScheduleAssignSkill

Help for this Page ?

Apex Class Detail

Edit

Delete

Download

Security

Show Dependencies

Name	ScheduleAssignSkill	Status	Active
Namespace Prefix		Code Coverage	0% (0/3)
Created By	T. Rupasree , 9/21/2025, 4:19 AM	Last Modified By	T. Rupasree , 9/21/2025, 4:19 AM

Class Body

Class Summary

Version Settings

Trace Flags

```

1  global class ScheduleAssignSkill implements Schedulable {
2
3      global void execute(SchedulableContext sc) {
4          // Run the batch to assign skills
5          AssignSkillBatch batch = new AssignSkillBatch();
6          Database.executeBatch(batch, 200);
7          System.debug('Scheduled Batch Apex executed successfully!');
8      }
9  }

```

Edit

Delete

Download

Security

Show Dependencies

3 To Schedule the Apex Job

Option 1: Developer Console (Execute Anonymous)

```
// Cron Expression: "0 0 23 * * ?" → Runs daily at 11 PM
String cronExp = '0 0 23 * * ?';
System.schedule('Daily Skill Assignment', cronExp, new ScheduleAssignSkill());
```

Option 2: Salesforce UI

1. Go to **Setup** → **Apex Classes** → **Schedule Apex** → **New**
2. Enter **Job Name** → e.g., `Daily Skill Assignment`
3. Select **Apex Class** → `ScheduleAssignSkill`
4. Define **Frequency** → daily, weekly, or monthly
5. Set **Start & End Dates** and **Preferred Time**
6. Click **Save**

Save Cancel

Error: Invalid Data.
Review all error messages below to correct your data.

Job Name

Apex Class 

Schedule Using ☒ Schedule Builder ☐ Cron Expression

Schedule Apex Execution

Frequency ☒ Weekly ☐ Monthly

Start [9/21/2025]

End [9/21/2025]

Preferred Start Time 
Error: You must enter a value

Exact start time will depend on job queue activity.

Error: Please select a preferred time

Save Cancel

4 How It Works

1. Salesforce runs the **ScheduleAssignSkill.execute()** method at the scheduled time
2. Inside it, the **AssignSkillBatch** is executed to process completed trainings
3. Skills are assigned automatically without any manual intervention

✓ Practical use for Your Project

- Use case:
- Send **weekly reminders** to candidates for pending training.

- Notify employers about candidate shortlists.
-

10. Future Methods

Future methods allows to:

- Run Apex **asynchronously**
- Avoid blocking the user interface
- Execute tasks **later** in a separate thread

Key Points:

- Annotated with `@future`
 - Must be **static**
 - Can only accept **primitive data types or collections** (no sObjects or complex objects)
 - Cannot return a value
-

1 Use Case in Your Project

Scenario: Send a notification email to the candidate **after a skill is assigned**.

- Object: `Candidate__c`
 - Trigger: After skill assignment
 - Action: Email candidate asynchronously using a future method
-

2 Example Future Method

```
public class CandidateNotification { @future public static void
sendSkillNotification(Id candidateId, String skillName) { Candidate__c cand
= [SELECT Id, Email__c, Student_Name__c FROM Candidate__c WHERE Id =
:candidateId LIMIT 1]; // Example: Just debug for now System.debug('Email
sent to ' + cand.Email__c + ' about new skill: ' + skillName); // In real
scenario, you can use Messaging.sendEmail() here } }
```

Apex Class

CandidateNotification

Apex Class Detail [Edit](#) [Delete](#) [Download](#) [Security](#) [Show Dependencies](#)

Name	CandidateNotification	Status	Active
Namespace Prefix		Code Coverage	0% (0/2)
Created By	T. Rupasree , 9/21/2025, 4:23 AM	Last Modified By	T. Rupasree

[Class Body](#) [Class Summary](#) [Version Settings](#) [Trace Flags](#)

```

1 public class CandidateNotification {
2
3     @future
4     public static void sendSkillNotification(Id candidateId, String skillName) {
5         Candidate__c cand = [SELECT Id, Email__c, Student_Name__c FROM Candidate__c WHERE
6
7         // Example: Just debug for now
8         System.debug('Email sent to ' + cand.Email__c + ' about new skill: ' + skillName);
9
10        // In real scenario, you can use Messaging.sendEmail() here
11    }
12 }

```

[Edit](#) [Delete](#) [Download](#) [Security](#) [Show Dependencies](#)

3 How to Call the Future Method

```
// After skill assignment Id candidateId = 'PUT_CANDIDATE_ID_HERE'; String
newSkill = 'Java Programming';
CandidateNotification.sendSkillNotification(candidateId, newSkill);
```

- This call **returns immediately**; the job runs asynchronously in the background

Enter Apex Code

```

1 Id candidateId = 'a02gL0000073v5nQAA';
2 String newSkill = 'Java Programming';
3
4 CandidateNotification.sendSkillNotification(candidateId, newSkill);
5

```

☒ Open Log [Execute](#) [Execute Highlighted](#)

4 Trigger Example for Your Project

You can call the future method **from a trigger** when a new `Skill__c` is inserted:

```
trigger SkillTrigger on Skill__c (after insert) { for(Skill__c s :
Trigger.new) { CandidateNotification.sendSkillNotification(s.Candidate__c,
s.Skill_Name__c); } }
```

- Every time a new skill is added, the **candidate gets a notification asynchronously**

Execution Log		
Timestamp	Event	Details
16:55:17:001	USER_INFO	[EXTERNAL]]005gL000005Dp8H rupasree287375@ageantforce.com (GMT-07:00) Pacific Daylight Time (America/Los_A
16:55:17:001	EXECUTION_ST...	
16:55:17:001	CODE_UNIT_ST...	[EXTERNAL]]execute_anonymous_apex
16:55:17:002	VARIABLE_SCO...	[1]]candidateId Id false false
16:55:17:002	VARIABLE_SCO...	[2]]newSkill String false false
16:55:17:002	HEAP_ALLOCATE	[95]]Bytes:3
16:55:17:002	HEAP_ALLOCATE	[100]]Bytes:152
16:55:17:002	HEAP_ALLOCATE	[417]]Bytes:408
16:55:17:002	HEAP_ALLOCATE	[430]]Bytes:408
16:55:17:002	HEAP_ALLOCATE	[317]]Bytes:6
16:55:17:002	HEAP_ALLOCATE	[EXTERNAL]]Bytes:5
16:55:17:002	STATEMENT_EX...	[1]]
16:55:17:002	STATEMENT_EX...	[1]]
16:55:17:002	HEAP_ALLOCATE	[1]]Bytes:18
16:55:17:002	HEAP_ALLOCATE	[1]]Bytes:4
16:55:17:003	VARIABLE_ASSI...	[1]]candidateId "a02gL0000073v5nQAA"
16:55:17:003	STATEMENT_EX...	[2]]
16:55:17:003	HEAP_ALLOCATE	[2]]Bytes:16
16:55:17:003	VARIABLE_ASSI...	[2]]newSkill "Java Programming"
16:55:17:003	STATEMENT_EX...	[4]]
16:55:17:003	HEAP_ALLOCATE	[4]]Bytes:21
16:55:17:003	HEAP_ALLOCATE	[4]]Bytes:21
16:55:17:003	HEAP_ALLOCATE	[4]]Bytes:4
16:55:17:003	HEAP_ALLOCATE	[4]]Bytes:4
16:55:17:003	HEAP_ALLOCATE	[4]]Bytes:2
16:55:17:003	HEAP_ALLOCATE	[4]]Bytes:6
16:55:17:003	HEAP_ALLOCATE	[68]]Bytes:5
16:55:17:003	HEAP_ALLOCATE	[74]]Bytes:5
16:55:17:003	HEAP_ALLOCATE	[82]]Bytes:7
16:55:17:003	HEAP_ALLOCATE	[EXTERNAL]]Bytes:12
16:55:17:003	HEAP_ALLOCATE	[EXTERNAL]]Bytes:12
16:55:17:038	CUMULATIVE_L...	
16:55:17:038	LIMIT_USAGE_...	(default)
16:55:17:000	LIMIT_USAGE_...	Number of SOQL queries: 0 out of 100

Benefits

- Runs **asynchronously**, so it doesn't slow down the user interface
- Can handle **multiple operations in the background**
- Simple to implement for **notifications, logging, or external API calls**
 - Use case: External API callouts to integrate with **training providers (Coursera/edX)**.

11. Exception Handling

Purpose:

- Catch and handle **runtime errors** gracefully
- Prevent your Apex code from **failing unexpectedly**
- Log errors for **debugging and auditing**

Key Concepts:

Term	Description
<code>try</code>	Block of code where errors might occur
<code>catch</code>	Block to handle the exception
<code>finally</code>	Optional block that always executes, used for cleanup
<code>throw</code>	Used to raise a custom exception

1 Use Case in our Project

Scenario:

- While assigning skills or creating jobs, some records might fail due to **missing required fields** or **duplicate values**.
 - Handle these errors gracefully and optionally **log them in a custom object** like `Error_Log__c`.
-

2 Example: Standard Exception Handling

```
public class SkillManagerSafe { public static void assignSkillSafe(Id candidateId, String skillName) { try { // Attempt to assign skill Skill__c newSkill = new Skill__c( Candidate__c = candidateId, Skill_Name__c = skillName ); insert newSkill; System.debug('Skill assigned: ' + skillName); } catch(DmlException e) { System.debug('Error inserting Skill: ' + e.getMessage()); // Optional: log error to a custom object Error_Log__c log = new Error_Log__c( Name = 'Skill Assignment Error', Description__c = e.getMessage() ); insert log; } finally { System.debug('Execution completed for assignSkillSafe method'); } } }
```

Apex Class

SkillManagerSafe

Help for this Page ?

Apex Class Detail

Edit

Delete

Download

Security

Show Dependencies

Name	SkillManagerSafe	Status	Active
Namespace Prefix		Code Coverage	0% (0/11)
Created By	T. Rupasree , 9/21/2025, 4:35 AM	Last Modified By	T. Rupasree , 9/21/2025, 4:35 AM

Class Body

Class Summary

Version Settings

Trace Flags

```
1 public class SkillManagerSafe {
2
3     public static void assignSkillSafe(Id candidateId, String skillName) {
4         try {
5             // Attempt to assign skill
6             Skill__c newSkill = new Skill__c(
7                 Candidate__c = candidateId,
8                 Skill_Name__c = skillName
9             );
10            insert newSkill;
11
12            System.debug('Skill assigned: ' + skillName);
13
14        } catch(DmlException e) {
15            System.debug('Error inserting Skill: ' + e.getMessage());
16
17            // Optional: log error to a custom object
18            Error_Log__c log = new Error_Log__c(
19                Name = 'Skill Assignment Error',
20                Description__c = e.getMessage()
21            );
22            insert log;
23        } finally {
24            System.debug('Execution completed for assignSkillSafe method');
25        }
26    }
27 }
```

Edit

Delete

Download

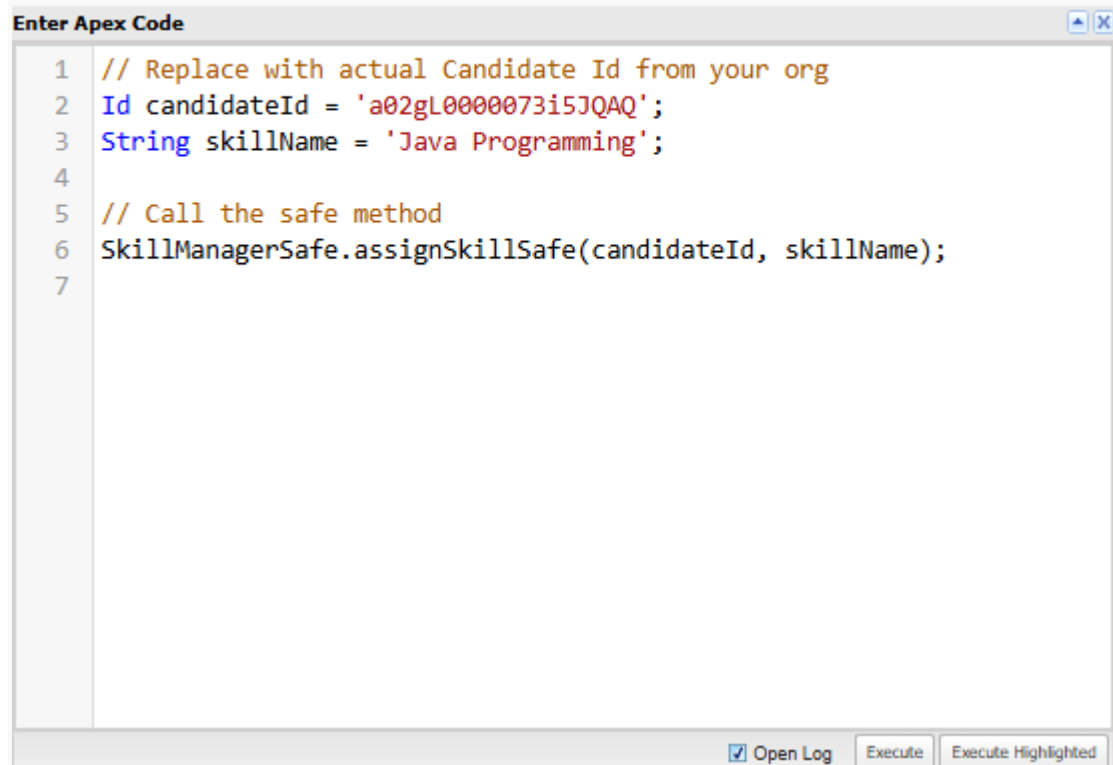
Security

Show Dependencies

3 Example: Throwing Custom Exceptions

```
public class JobApplicationHandler { public class JobApplicationException extends Exception {} public static void applyCandidate(Id candidateId, Id jobId) { // Check if application already exists List<Job_Application__c>
```

```
existingApps = [ SELECT Id FROM Job_Application__c WHERE Candidate__c =
:candidateId AND Job__c = :jobId ]; if(!existingApps.isEmpty()) { // Throw
custom exception throw new JobApplicationException('Candidate has already
applied for this job. '); } // Insert application if no error
Job_Application__c newApp = new Job_Application__c( Candidate__c =
candidateId, Job__c = jobId, Application_Status__c = 'Applied' ); insert
newApp; } }
```



```
1 // Replace with actual Candidate Id from your org
2 Id candidateId = 'a02gL0000073i5JQAQ';
3 String skillName = 'Java Programming';
4
5 // Call the safe method
6 SkillManagerSafe.assignSkillSafe(candidateId, skillName);
7
```

- **Use Case:** Prevent duplicates and notify users/programmatically handle errors

Execution Log					
Timestamp	Event	Details			
17:07:56:003	METHOD_ENTRY	[6] 01pgL000005h5m9 SkillManagerSafe.assignSkillSafe(Id, String)			
17:07:56:003	VARIABLE_SCO...	[3] candidateId Id false false			
17:07:56:003	VARIABLE_ASSI...	[3] candidateId "a02gL0000073i5JQAQ"			
17:07:56:003	VARIABLE_SCO...	[3] skillName String false false			
17:07:56:003	VARIABLE_ASSI...	[3] skillName "Java Programming"			
17:07:56:003	STATEMENT_EX...	[3]			
17:07:56:003	STATEMENT_EX...	[4]			
17:07:56:003	STATEMENT_EX...	[4]			
17:07:56:003	STATEMENT_EX...	[6]			
17:07:56:003	HEAP_ALLOCATE	[6] Bytes:4			
17:07:56:003	VARIABLE_ASSI...	[6] this.Candidate__c "a02gL0000073i5JQAQ" 0x3f35c2e1			
17:07:56:003	VARIABLE_ASSI...	[6] this.Skill_Name__c "Java Programming" 0x3f35c2e1			
17:07:56:003	VARIABLE_SCO...	[6] newSkill Skill__c true false			
17:07:56:003	VARIABLE_ASSI...	[6] newSkill {"Candidate__c":"a02gL0000073i5JQAQ","Skill_Name__c":"Java Programming"} 0x3f35c2e1			
17:07:56:003	STATEMENT_EX...	[10]			
17:07:56:003	HEAP_ALLOCATE	[10] Bytes:8			
17:07:56:003	DML_BEGIN	[10] Op:Insert Type:Skill__c Rows:1			
17:07:56:003	HEAP_ALLOCATE	[EXTERNAL] Bytes:8			
17:07:56:037	DML_END	[10]			
17:07:56:037	STATEMENT_EX...	[12]			
17:07:56:037	HEAP_ALLOCATE	[12] Bytes:16			
17:07:56:037	HEAP_ALLOCATE	[12] Bytes:32			
17:07:56:037	USER_DEBUG	[12] DEBUG Skill assigned: Java Programming			
17:07:56:037	STATEMENT_EX...	[23]			
17:07:56:037	STATEMENT_EX...	[24]			
17:07:56:037	HEAP_ALLOCATE	[24] Bytes:46			
17:07:56:037	USER_DEBUG	[24] DEBUG Execution completed for assignSkillSafe method			
17:07:56:037	METHOD_EXIT	[6] 01pgL000005h5m9 SkillManagerSafe.assignSkillSafe(Id, String)			
17:07:56:037	SYSTEM_MODE...	false			
17:07:56:059	CUMULATIVE_L...				
17:07:56:059	LIMIT_USAGE...	(default)			
17:07:56:000	LIMIT_USAGE	Number of SOQL queries: 0 out of 100			
☐ This Frame ☐ Executable ☐ Debug Only ☐ Filter Click here to filter the log					
Logs Tests Checkpoints Query Editor View State Progress Problems					
User	Application	Operation	Time	Status	Read
T. Rupasree	Unknown	/services/data/v64.0/too...	9/21/2025, 5:07:56 PM	Success	
T. Rupasree	Unknown	/services/data/v64.0/too...	9/21/2025, 5:07:37 PM	Success	Unread
T. Rupasree	Browser	/setup/build/listApexClas...	9/21/2025, 5:06:22 PM	Success	Unread

- Example: Candidate profile missing mandatory details.
- `try-catch-finally` ensures errors don't break bulk operations.

12. Test Classes

- Test skill assignment, enrollment automation, and job validation.
- Ensures **75%+ code coverage** for deployment.

Example:

```
@isTest private class SkillManagerTest { @isTest static void
testAssignSkill(){ Candidate__c cand = new Candidate__c(Name='Test User',
Email__c='test@test.com'); insert cand; SkillManager.assignSkill(cand.Id,
'Apex Development'); List<Skill__c> skills = [SELECT Name FROM Skill__c
WHERE Candidate__c = :cand.Id]; System.assertEquals('Apex Development',
skills[0].Name); } }
```

Key Points:

- Must be annotated with `@isTest`
 - Should **cover at least 75% of Apex code** for production deployment
 - Can **create test data** in the class (don't rely on existing org data)
 - Can test both **positive and negative scenarios**
-

1 Test Class for SkillManagerSafe

```
@isTest public class SkillManagerSafeTest { @isTest static void
testAssignSkillSafeSuccess() { // 1 Create test Candidate Candidate__c
testCandidate = new Candidate__c( Email__c = 'testuser@example.com',
ContactNumber__c = '9001234567', Student_Name__c = 'a01gL000000XXXX' //
Replace with actual Student Id or create test Student ); insert
testCandidate; // 2 Call the safe method Test.startTest();
SkillManagerSafe.assignSkillSafe(testCandidate.Id, 'Java Programming');
Test.stopTest(); // 3 Verify Skill record was created List<Skill__c> skills
= [SELECT Id, Skill_Name__c, Candidate__c FROM Skill__c WHERE Candidate__c
= :testCandidate.Id]; System.assertEquals(1, skills.size(), 'One skill
record should be created'); System.assertEquals('Java Programming',
skills[0].Skill_Name__c, 'Skill name should match'); } @isTest static void
testAssignSkillSafeException() { // 1 Call method with invalid Candidate Id
to force exception Test.startTest(); try {
SkillManagerSafe.assignSkillSafe('invalidId', 'Python Programming'); }
catch (Exception e) { System.debug('Expected exception caught: ' +
e.getMessage()); } Test.stopTest(); // 2 Verify error was logged
List<Error Log c> logs = [SELECT Id, Name, Description c FROM
Error_Log__c WHERE Name = 'Skill Assignment Error'];
System.assert(logs.size() > 0, 'Error log should be created'); } }
```

Class Body Class Summary Version Settings Trace Flags

```

1  @isTest
2  public class SkillManagerSafeTest {
3
4      @isTest static void testAssignSkillSafeSuccess() {
5          // 1 Create test Candidate
6          Candidate__c testCandidate = new Candidate__c(
7              Email__c = 'testuser@example.com',
8              ContactNumber__c = '9001234567',
9              Student_Name__c = 'a01gL000000XXXX' // Replace with actual Student Id or create test Student
10         );
11         insert testCandidate;
12
13         // 2 Call the safe method
14         Test.startTest();
15         SkillManagerSafe.assignSkillSafe(testCandidate.Id, 'Java Programming');
16         Test.stopTest();
17
18         // 3 Verify Skill record was created
19         List<Skill__c> skills = [SELECT Id, Skill_Name__c, Candidate__c FROM Skill__c WHERE Candidate__c = :testCandidate.Id];
20         System.assertEquals(1, skills.size(), 'One skill record should be created');
21         System.assertEquals('Java Programming', skills[0].Skill_Name__c, 'Skill name should match');
22     }
23
24     @isTest static void testAssignSkillSafeException() {
25         // 1 Call method with invalid Candidate Id to force exception
26         Test.startTest();
27         try {
28             SkillManagerSafe.assignSkillSafe('invalidId', 'Python Programming');
29         } catch (Exception e) {
30             System.debug('Expected exception caught: ' + e.getMessage());
31         }
32         Test.stopTest();
33
34         // 2 Verify error was logged
35         List<Error_Log__c> logs = [SELECT Id, Name, Description__c FROM Error_Log__c WHERE Name = 'Skill Assignment Error'];
36         System.assertEquals(logs.size() > 0, 'Error log should be created');
37     }
38 }

```

[Edit](#) [Delete](#) [Download](#) [Run Test](#) [Show Dependencies](#)

2 Steps to Run the Test

1. Go to **Setup** → **Apex Classes** → click **“Run Tests”**
2. Select `SkillManagerSafeTest`
3. Click **Run**
4. Check **Test Results**:
 - `testAssignSkillSafeSuccess` → should **pass**
 - `testAssignSkillSafeException` → should **pass**, confirms **exception**

handling works

We Already implemented in before processes

13. Asynchronous Processing

- **Skill Development Side:** Bulk process course enrollments.
- **Employment Side:** Bulk process job applications during campus drives.

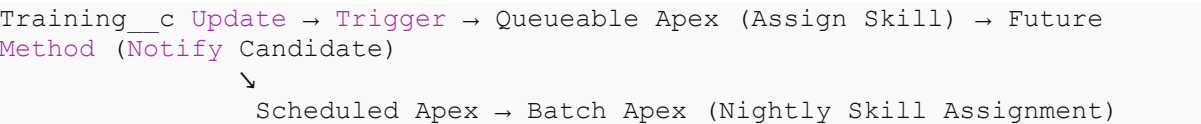
Salesforce provides multiple ways to execute operations **in the background** without blocking the user interface. In your project, you used:

Method	Use Case in Your Project	Key Points
Batch Apex	Assign skills to all candidates who completed training	Handles large datasets (50M+ records)
Queueable Apex	Assign skills as soon as a training is completed	Can chain jobs , supports complex objects
Scheduled Apex	Run skill assignments daily or send notifications	Schedule batch or queueable jobs at a specific time
Future Methods	Send emails/notifications asynchronously	Simple, fire-and-forget, cannot return values

Practical Flow for our Project

1. **Candidate completes training** → trigger fires
2. **Queueable Apex** assigns skill asynchronously
3. **Future Method** sends notification email to candidate
4. **Batch Apex** can process **all trainings in bulk nightly**
5. **Scheduled Apex** runs Batch Apex automatically every day

Visual Flow:



2 Best Practices for Asynchronous Processing

- **Using Queueable instead of Future** for complex objects
 - **Chain Queueables** for sequential operations
 - **Batch Apex** for bulk operations over thousands/millions of records
 - Always include **Test Classes** for async methods:
 - Use `Test.startTest()` and `Test.stopTest()` to execute async jobs in tests
 - **Handle exceptions** inside async jobs (try-catch) to avoid silent failures
 - **Use limits wisely:**
 - Queueable: 50 jobs in queue at a time
 - Batch: can process 50 million records
 - Future: 50 calls per transaction
-

3 Example Combined Scenario

```
trigger TrainingTrigger on Training__c (after update) { for(Training__c tr
: Trigger.new) { Training__c oldTr = Trigger.oldMap.get(tr.Id);
if(tr.Training_Status__c == 'Completed' && oldTr.Training_Status__c !=
'Completed') { // Assign skill asynchronously System.enqueueJob(new
AssignSkillQueueable(tr.Id)); } } } // Scheduled Apex runs nightly global
class ScheduleAssignSkill implements Schedulable { global void
execute(SchedulableContext sc) { Database.executeBatch(new
AssignSkillBatch(), 200); } } // Future method for notifications public
class CandidateNotification { @future public static void
sendSkillNotification(Id candidateId, String skillName) { // Send email or
debug System.debug('Notification sent to Candidate: ' + candidateId + ' for
skill: ' + skillName); } }
```

Summary

- **Queueable** → real-time async processing
 - **Batch Apex** → bulk processing (large datasets)
 - **Scheduled Apex** → time-based automation
 - **Future Methods** → lightweight async (notifications/logs)
 - **Exception Handling** → ensures safe execution
 - **Test Classes** → verify all scenarios
-

This completes **Phase 5: Apex Programming** for your **Skill Development & Employment Portal**.

◆ Phase 5 Deliverables

- Apex Classes for **skill management, enrollment, and job applications**.
 - Triggers with **handler classes**.
 - Batch, Queueable, Scheduled Apex jobs.
 - Exception handling with logs.
 - Unit test classes with high code coverage.
-

◆ How Phase 5 Connects Skill Development & Employment

- ✓ Candidates can **enroll in training programs** → Skills automatically updated.
- ✓ Employers can **search/filter candidates** based on acquired skills.
- ✓ Job applications are **auto-validated** against required skills.
- ✓ Bulk operations (batch jobs) ensure scalability for large user bases.

This phase marks the **fusion** of **Skill Development** and **Employment Management**, making the portal more **intelligent, automated, and scalable**.